



Summer Internship Report

Tensor Network: A Study of Matrix Product States

Submitted to

Prof. Sudipto Singha Roy

Department of Physics

Indian Institute of Technology (ISM) Dhanbad

Submitted By

Moirangthem Denison Meitei
M.S. in Quantum Technology
Indian Institute of Science Education
and Research Pune

Internship Conducted At

Department of Physics
Indian Institute of Technology
(Indian School of Mines) Dhanbad

July 2026

DECLARATION

I, **Moirangthem Denison Meitei**, hereby declare that the project report entitled “**Tensor Network: A study of Matrix Product States**” submitted to the **Summer Research Internship Program 2026** is a result of my original research work carried under the guidance of **Dr. Sudipto Singh Roy**, Assistant Professor, Department of Physics, Indian Institute of Technology (Institute of Mines), Dhanbad - Jharkhand 826007, India.

Moirangthem Denison Meitei

ID no. 26SR0099

16 July, 2026

Dhanbad

Acknowledgements

I would like to express my sincere gratitude to **Prof. Sudipto Singha Roy**, Department of Physics, Indian Institute of Technology (Indian School of Mines) Dhanbad, for providing me with the opportunity to undertake this summer internship under his guidance. His insightful discussions, constant encouragement, and valuable suggestions greatly enhanced my understanding of tensor network methods and their applications in quantum many-body physics. I am truly grateful for his guidance throughout the course of this project.

I am equally thankful to **Mr. Arkaparva Sil**, Ph.D. scholar under Prof. Roy, for his continuous support and guidance during the internship. His patience in answering my questions, helping me deepen my understanding of the tensor network theory, implementation of tensor network algorithms, and assisting me with numerical simulations using the ITensor library proved invaluable to this work.

I would also like to thank the **Department of Physics, Indian Institute of Technology (Indian School of Mines) Dhanbad** for providing an excellent academic environment and the necessary resources to carry out this project.

I gratefully acknowledge the use of **Google Gemini** as a learning companion for exploring and developing a deeper understanding of concepts related to tensor networks, matrix product states, and their mathematical foundations. Its explanations served as a valuable supplement while studying textbooks and research articles.

I also acknowledge the use of **OpenAI's ChatGPT** as an academic writing and productivity assistant. ChatGPT assisted in improving the language, Latex typesetting and presentation of this report, refining explanations, and generating several Tensor Network diagrams in the **TikZ** package based on my specifications. All technical content, mathematical derivations, numerical implementations, interpretations, and conclusions presented in this report were written by me using standard textbooks, research articles, and my own understanding.

Finally, I would like to express my sincere gratitude to the **Indian Institute of Science Education and Research (IISER) Pune** for providing me with a strong academic foundation and encouraging me to pursue this internship. I am also thankful to my teachers, friends, and family for their constant support, encouragement, and motivation throughout this journey.

Abstract

This report presents a study of the theoretical foundations of tensor networks with a primary focus on Matrix Product States. The concepts of tensor network representations, canonical forms of MPS, and the role of entanglement in one-dimensional systems are discussed. The report also introduces variational optimization and imaginary time evolution as approaches for approximating ground states of quantum many-body systems.

The exponential growth of the Hilbert space with system size makes the numerical study of quantum many-body systems increasingly difficult using exact diagonalization methods. Tensor network techniques provide an efficient framework for addressing this challenge by exploiting the entanglement structure of low-dimensional quantum systems. Among these methods, Matrix Product States (MPS) offer a compact representation of one-dimensional quantum states and form the basis of several powerful numerical algorithms.

To demonstrate the practical application of these methods, the Density Matrix Renormalization Group (DMRG) algorithm is implemented using the ITensor library in Julia and applied to the one-dimensional Transverse Field Ising Model. The numerical simulation using the DMRG algorithm successfully reproduced the expected behavior of the model across the quantum phase transition and illustrated the effectiveness of MPS-based methods for studying one-dimensional quantum systems. Overall, this work provides both a theoretical introduction to Matrix Product States and a practical implementation of tensor network algorithms for ground-state calculations.

Contents

1	Introduction	1
1.1	Why are we interested in Tensor Networks?	1
1.1.1	Gottesman-Knill theorem	1
1.1.2	Role of Tensor Network	2
1.2	Entanglement Entropy and the Area Law	2
1.3	Families of Tensor Network States	3
1.4	Ground-State Optimization Algorithms	4
1.5	Objectives and Scope of the Present Work	5
2	Tensor Network and its representation	6
2.1	Brief description about Tensors	6
2.1.1	Definition	6
2.1.2	Rank of tensor	7
2.2	Tensor Network	7
2.2.1	Tensor Network Diagram	8
2.2.2	Construction of wave-function with small pieces	10
2.2.3	Estimation of total number of parameters in a TN	11
2.3	Components of the Tensor Network	12
3	Matrix Product State (MPS) and its Canonical Form	14
3.1	What are MPS?	14
3.2	Canonical Form of MPS	15
3.2.1	Schmidt Decomposition	16
3.2.2	Construction of the canonical form	17
3.2.3	Canonical Forms of finite MPS	20
3.2.4	Canonical Form of an Infinite MPS	20
3.2.5	Properties of the Canonical Form of an MPS	22
3.3	Correlation in MPS	23
3.4	Exact Calculation of the expectation values	25
3.5	Fundamental Properties of MPS	26
3.5.1	Translational Invariance and the Thermodynamic Limit	26
3.5.2	Representational Power of MPS	27
3.5.3	One-dimensional area-law	27
3.6	Matrix Product Operator	27
4	Finding Ground States using MPS	29
4.1	Variational Optimization	29
4.1.1	Density Matrix Renormalization Algorithm (DMRG)	31
4.2	Imaginary Time Evolution	31

4.2.1	Time Evolving Block Decimation	34
4.3	Stability and the Normalization matrix $\mathcal{N}$	35
5	Application of MPS to the Transverse Field Ising model for benchmarking	36
5.1	The Transverse Field Ising Model	36
5.2	Benchmarking using ITensor	36
5.3	Challenges faced in Benchmarking	37
5.4	Magnetization and the GHZ states	37
5.5	Numerical Result	38
5.6	Discussion	39
A	Suzuki Trotter Expansion	41
B	Implementing ITensor in Julia	43
	Bibliography	45

Chapter 1

Introduction

The study of quantum many-body systems lies at the heart of condensed matter physics, quantum chemistry, and quantum information science. Understanding the properties of these systems, particularly their ground states and low-energy excitations, is essential for explaining a wide range of physical phenomena, from magnetism and superconductivity to quantum phase transitions. Despite the apparent simplicity of their microscopic interactions, solving quantum many-body systems exactly remains one of the most challenging problems in modern physics.

The primary difficulty arises from the exponential growth of the Hilbert space with the size of the system. Let us consider a simple system with one-dimensional spin- $\frac{1}{2}$ chain consisting of N sites. Since each spin has two possible basis states, the dimension of the corresponding Hilbert space is

$$\dim(\mathcal{H}) = 2^N. \quad (1.1)$$

Consequently, the number of coefficients required to describe a general quantum state,

$$|\psi\rangle = \sum_{i_1, i_2, \dots, i_N} C_{i_1 i_2 \dots i_N} |i_1 i_2 \dots i_N\rangle, \quad (1.2)$$

also grows exponentially with the system size. Even for moderately large values of N , storing these coefficients exceeds the memory capacity of modern classical computers, making exact numerical simulations computationally infeasible.

1.1 Why are we interested in Tensor Networks?

The exponential growth of the Hilbert space naturally raises the question of whether classical simulation of quantum systems is fundamentally impossible. Interestingly, this is not always the case. Although an arbitrary quantum state generally requires an exponential amount of information to describe, many physically relevant quantum states possess an internal structure that can be exploited to achieve an efficient classical representation.

1.1.1 Gottesman-Knill theorem

The Gottesman–Knill theorem[1] states that quantum circuits composed exclusively of Clifford gates (Hadamard, Pauli, Phase and CNOT gates), computational basis state preparation and Pauli measurements together with the probability of classical control conditioned

on the outcome of such measurements can be efficiently simulated on a classical computer despite generating highly entangled quantum states. This remarkable result shows that entanglement alone is not sufficient to characterize the computational complexity of a quantum system. Instead, the structure of the entanglement plays an equally important role.

The Gottesman–Knill theorem suggests that only a restricted subset of quantum states exhibits the full exponential complexity associated with the Hilbert space. In particular, the ground states of local Hamiltonians encountered in condensed matter physics possess a highly organized pattern of quantum correlations rather than arbitrary entanglement. Consequently, it becomes unnecessary to represent every possible state in the Hilbert space. Instead, one seeks an efficient representation that captures only the physically relevant corner of the Hilbert space.

1.1.2 Role of Tensor Network

Tensor Networks provide precisely such a representation. By decomposing the exponentially large coefficient tensor into a network of smaller interconnected tensors, they exploit the underlying entanglement structure of quantum many-body states while dramatically reducing the number of variational parameters. This enables the study of systems far beyond the reach of exact diagonalization methods.

The effectiveness of Tensor Networks, however, is not merely a consequence of tensor decomposition. Their success relies on a fundamental physical property exhibited by the ground states of local Hamiltonians, namely the area law of entanglement. Understanding this principle is therefore essential for explaining why Tensor Networks provide such an efficient description of quantum many-body systems, which is the subject of the next section.

1.2 Entanglement Entropy and the Area Law

In the previous section we show that the difficulty of simulating quantum many-body systems arises from the exponential growth of the Hilbert space and the amount of quantum entanglement present in the system. Many important Hamiltonians in nature have local interactions between different particles. They have nearest or next-to-nearest neighbours interaction. In particular, the ground states of local Hamiltonians possess a remarkably simple entanglement structure, which is the fundamental reason why tensor network methods are so successful. Consider a quantum system divided into two complementary subsystems A and B ,

$$\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B.$$

If the composite system is in a pure state $|\psi\rangle$, the reduced density matrix describing subsystem A is obtained by tracing over the degrees of freedom of subsystem B ,

$$\rho_A = \text{tr}_B (|\psi\rangle \langle\psi|). \quad (1.3)$$

The amount of entanglement between the two subsystems is quantified by the von Neumann entropy,

$$S(\rho_A) = -\text{tr} (\rho_A \log \rho_A). \quad (1.4)$$

A product state has zero entanglement entropy, whereas an entangled state possesses a non-zero entropy. Thus, the entanglement entropy measures how strongly the two subsystems

are quantum mechanically correlated. For a generic quantum state chosen from the Hilbert space, the entanglement entropy grows proportionally to the volume of the subsystem,

$$S(A) \propto A, \quad (1.5)$$

where A denotes the number of sites contained in subsystem A . This behaviour is known as the *volume law*. Consequently, generic quantum states require an exponentially large amount of information to represent, making efficient classical simulation practically impossible. Remarkably, the physically relevant states encountered in nature have a unique property. The low energy states of realistic Hamiltonians obey the *area law of entanglement*[2]. This is because they are heavily constrained by the locality. Instead of scaling with the volume of the subsystem, the entanglement entropy now grows only with the size of the boundary separating the two subsystems,

$$S(A) \propto \partial A, \quad (1.6)$$

where ∂A denotes the boundary area between the bipartition.

In one dimensional space the boundary between two contiguous blocks consists of only a constant number of bonds. Consequently, the area law reduces to

$$S(L) \sim \text{constant}, \quad (1.7)$$

for sufficiently large block size L . Thus, the entanglement entropy saturates instead of increasing with the system size. This behaviour is characteristic of the ground states of one-dimensional gapped local Hamiltonians.

The area law reveals that physically relevant quantum states occupy only a tiny corner of the exponentially large Hilbert space. Therefore, it is unnecessary to represent arbitrary quantum states. Instead, one seeks a representation that efficiently captures only those states whose entanglement satisfies the area law.

Tensor Networks are constructed precisely for this purpose. By decomposing the exponentially large coefficient tensor into a network of smaller interconnected tensors, they efficiently encode the limited entanglement structure exhibited by low-energy states. The Renormalization Group (RG) methods utilizes tensor network and targets to identify and keep tracks of the relevant degrees of freedom to describe the system. This enables an accurate approximation of many-body ground states while avoiding the exponential complexity associated with the full Hilbert space.

Among the various tensor network representations, Matrix Product States (MPS) are particularly well suited for one-dimensional quantum systems. Their effectiveness is closely connected to the area law and the finite correlation length exhibited by the ground states of gapped local Hamiltonians. This connection is discussed in the next section.

1.3 Families of Tensor Network States

Tensor Networks constitute a broad family of variational ansätze designed to efficiently represent quantum many-body states. Although they share the common idea of decomposing a high-rank tensor into a network of lower-rank tensors, different tensor network structures are better suited for different physical systems and spatial dimensions. The choice of tensor network therefore depends on the underlying geometry of the problem as well as the entanglement structure of the quantum state.

One of the simplest and most successful tensor network ansätze is the *Matrix Product State*

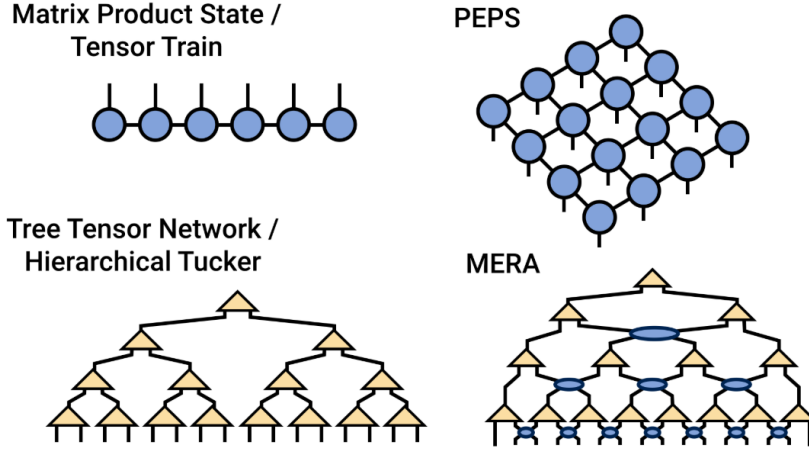


Figure 1.1: Families of Tensor Network (source: TensorNetwork.org)

(MPS), which provides an efficient representation of one-dimensional quantum systems. Owing to its close connection with the area law of entanglement, MPS has become the standard framework for studying ground-state properties of one-dimensional local Hamiltonians. For higher-dimensional systems, the natural generalization of MPS is the *Projected Entangled Pair State* (PEPS), where tensors are arranged according to the geometry of the underlying lattice. Other tensor network architectures have also been developed to efficiently describe different classes of quantum systems. For example, *Tree Tensor Networks* (TTN) organize tensors in a hierarchical tree structure and are particularly useful for systems with naturally hierarchical correlations, while the *Multi-scale Entanglement Renormalization Ansatz* (MERA) incorporates disentangling transformations and is especially well suited for describing critical systems exhibiting scale invariance.

A schematic illustration of some commonly used tensor network architectures is shown in Fig. 1.1. Among these various tensor network representations, the present work focuses exclusively on Matrix Product States and the numerical algorithms developed for their optimization. The remaining challenge is that an MPS representation alone does not determine the ground state of a given Hamiltonian. Efficient optimization algorithms are therefore required to determine the tensors constituting the MPS approximation.

1.4 Ground-State Optimization Algorithms

As the Matrix Product States provide an efficient representation of one-dimensional quantum many-body states, they are effective to determine the ground state of a given Hamiltonian. The tensors constituting the MPS must be optimized so that the corresponding quantum state minimizes the energy expectation value along with the constraint the norm of the state is unity. This optimization forms the central problem in tensor network simulations, and several numerical algorithms have been developed to efficiently search the exponentially large Hilbert space.

Among the most successful approaches are the *Density Matrix Renormalization Group* (DMRG)[2][3][4][5] and the *Time-Evolving Block Decimation* (TEBD)[2][4][5] algorithm. DMRG is based on the variational principle, where the tensors of the MPS are optimized iteratively by minimizing the expectation value of the Hamiltonian one site (or two sites) at a time. Owing to its high accuracy and numerical stability, DMRG has become the standard

method for determining the ground states of one-dimensional local Hamiltonians.

An alternative approach is provided by imaginary time evolution, which forms the basis of the TEBD algorithm. Starting from an initial state having a finite overlap with the ground state, repeated application of the imaginary time evolution operator exponentially suppresses the contributions from excited states, thereby slowly projecting the state onto the ground state. In practice, the evolution operator is decomposed into a sequence of local two-site gates using the Suzuki-Trotter expansion, allowing the state to be updated efficiently while preserving the MPS structure.

Both DMRG and TEBD exploit the efficient representation offered by Matrix Product States and have become indispensable tools for investigating strongly correlated quantum many-body systems. The mathematical formulation and implementation of these algorithms are presented in detail in Chapter 4. Although both DMRG and TEBD are introduced as representative ground-state algorithms, the numerical implementation presented in this work primarily employs the DMRG algorithm.

1.5 Objectives and Scope of the Present Work

The primary objective of this work is to investigate the application of Matrix Product States for the numerical simulation of one-dimensional quantum many-body systems. Particular emphasis is placed on understanding both the theoretical foundations of the MPS formalism and its practical implementation using the ITensor[4][5] library in Julia.

As a representative model, the Transverse Field Ising Model (TFIM) is considered throughout this work. The study focuses on determining the ground-state properties of the system using the Density Matrix Renormalization Group (DMRG) algorithm. To assess the performance of the MPS approach, several physical quantities are computed, including the ground-state energy, bipartite entanglement entropy, local expectation values, and two-point correlation functions. These quantities are further compared for different system sizes and for both open and periodic boundary conditions.

In addition to benchmarking the numerical implementation against established results, this work also investigates the practical challenges encountered during the optimization process. In particular, the limitations associated with periodic boundary conditions and the convergence difficulties arising in certain parameter regimes are examined. Strategies for improving the convergence of the optimization, including adiabatic continuation of the Hamiltonian parameters, are also explored where appropriate.

The theoretical concepts introduced in the subsequent chapters provide the foundation for these numerical investigations, while the implementation details and results are presented in the final chapter.

Chapter 2

Tensor Network and its representation

Before going into tensor network, we need to first understand tensor and its properties. Tensors are fundamental in physics especially to the areas like General Relativity, Electrodynamics, Classical and Quantum Field theories, Condensed matter, High Energy Physics, Statistical Physics, Quantum-many body physics etc. Tensors provide a compact mathematical framework for representing multidimensional data and multilinear maps. In modern computational physics, they play a central role in describing many-body quantum systems through tensor network methods. We briefly review the basic concepts of tensors highlighting their significance in Physics and how we use them in computing.

2.1 Brief description about Tensors

2.1.1 Definition

A tensor in the language of computer science or programming is a **multi-dimensional array**. To call items from the multi-dimensional array, we need indices and to each unique index a numerical value is associated or stored in it. For example, a matrix is a nested array of two arrays. Suppose $A = \begin{bmatrix} 1 & 1 \\ 4 & 5 \end{bmatrix}$ which in the form an array can be written as $[[1,1],[4,5]]$ and to access each value of A we need indices i and j such that A_{ij} represents an element of A with $i, j = 1, 2$.

The definition of a tensor typically used in physics is that a tensor is **a quantity that transforms like a tensor**. Although it sounds quite absurd, it is a self contained definition where we need to look into how a tensor transforms. The tensor transformation[6] is defined into two ways as:

- Covariant tensor:

$$T'_i = \sum_k \frac{\partial x^k}{\partial x'^i} T_k \quad (2.1)$$

- Contravariant tensor:

$$T'^i = \sum_k \frac{\partial x'^i}{\partial x^k} T_k \quad (2.2)$$

Any quantity following the above transformation rules will be considered as a tensorial quantity. Here, we can drop the summation symbol in the above equations and follow a

simple rule that if an index is repeated twice, we had to perform summation over that index. This is the **Einstein's summation convention**. The index which is repeated twice is called dummy index and those indices which are not repeated are called free index. For example,

$$A_{ij} = \sum_k P_{ik} Q_{kj} = P_{ik} Q_{kj} \quad (2.3)$$

The above equation is the matrix multiplication written with Einstein summation convention. Here, we are performing index contraction which is performing sum over a repeated index which is k. In strict tensor rule, we can have one single index repeated at most twice and that will perform contraction otherwise it will be an open index.

This set of definitions is strictly followed when we deal with general tensor where curved spaces are dealt. In the flat Euclidean space we have the Cartesian tensors, where both transformations are equivalent. For our purpose, we do not require the strict rules of general tensor and Einstein's summation convention. We will stick to normal definition where a tensor is a multi-dimensional array along with the freedom to use same index multiple times and follow Cartesian tensor formalism.

2.1.2 Rank of tensor

The rank of a tensor is simply defined as how many free indices does a tensor has. A tensor of rank 0 is a scalar (s), a tensor of rank 1 is a vector (V_i), a tensor of rank 2 is a matrix (M_{ij}) and higher ranked tensors are just called with their ranks e.g a tensor of rank 3 say T_{ijk} is called 3^{rd} ranked tensor. However, it is to be noted that not all vectors and matrices are 1^{st} and 2^{nd} ranked tensors because it is not necessary that every vectors and matrices satisfies (2.1) and (2.2) respectively but the inverse is true. This follows similarly for higher ranked tensors also.

The rank of a tensor becomes a lot more intuitive by the graphical notation which will be introduced in the following section. Each tensor is represented by a big circle and its index is represented by a leg. The number of legs a tensor has is what the rank of the tensor is.

2.2 Tensor Network

Having introduced the basic concept of tensors, we now turn our attention to tensor networks. A tensor network (TN) is a collection of tensors whose indices are connected through contraction of indices according to some pattern. Instead of representing a high-rank tensor as a single object, a tensor network decomposes it into several smaller tensors that are interconnected through contracted indices. This decomposition provides an efficient representation of high-dimensional data and forms the basis of modern tensor network methods. The fundamental operation in a tensor network is the contraction of tensor indices. Tensor contraction can be regarded as a generalization of matrix multiplication, where one or more common indices between tensors are summed over. For example,

$$C_{ij} = \sum_{\alpha=1}^D A_{i\alpha} B_{\alpha j} \quad (2.4)$$

Here, the tensors A and B are both rank-2 tensors. The index α common to both and is summed over, producing a new tensor C with two open indices i and j. Since the resulting tensor has two free indices, C is also a rank-2 tensor.

Let us consider a complicated example,

$$F_{ijkl} = \sum_{\alpha, \beta, \gamma, \delta, \mu=1}^D A_{i\alpha\beta\mu} B_{\beta\delta\gamma j} C_{k\alpha\delta} E_{\gamma l\mu} \quad (2.5)$$

In this case, five indices are contracted, while the indices i , j , k , and l remain open. Consequently, the resulting tensor F is a rank-4 tensor.

2.2.1 Tensor Network Diagram

In the last example, we observe that the algebraic expression appears cumbersome due to the large number of indices. However, its graphical representation is remarkably simple. This motivates the introduction of tensor network diagrams.

Tensor network diagrams provide an intuitive graphical language for representing tensors and the operations performed on them. Instead of manipulating lengthy tensor equations filled with multiple indices, one can represent the same mathematical expressions using simple diagrams, making the underlying structure of the computation much easier to visualize. In this graphical notation, a tensor network consists of a collection of tensors connected by lines. Each tensor is represented by a geometric shape (commonly a circle or square), while each line emerging from the tensor corresponds to one of its indices. Lines connecting two tensors represent contracted indices, whereas lines that remain unconnected correspond to open indices of the tensor network.

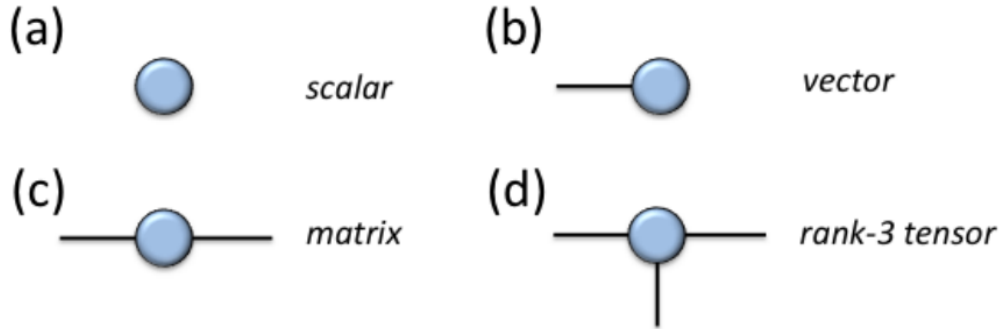
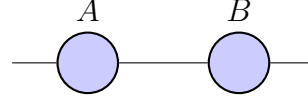


Figure 2.1: Graphical representation of tensors of different ranks (taken from Ref [2]).

We can see in the following examples how TN diagrams allow us to handle with complicated expressions in a visual way:

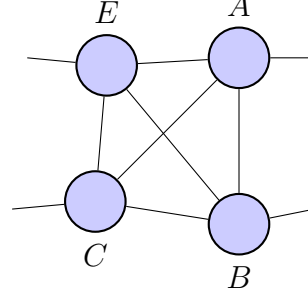
(i) Matrix product:

$$C_{ij} = \sum_{\alpha} A_{i\alpha} B_{\alpha j}$$



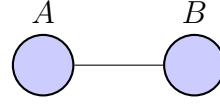
(ii) Contraction of four tensors:

$$F_{ijkl} = \sum_{\alpha, \beta, \gamma, \delta, \mu=1}^D A_{\alpha\beta\delta i} B_{\beta j \mu} C_{\delta \nu \mu k} E_{\nu l \alpha}$$



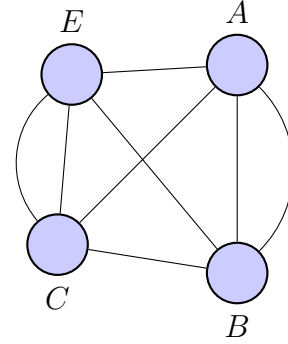
(iii) Inner product of two vectors:

$$G = \sum_{\alpha}^D A_{\alpha} B_{\alpha}$$



(iv) Contraction of four tensors into a scalar:

$$F = \sum_{\alpha, \beta, \gamma, \delta, \mu, i, k=1}^D A_{\alpha\beta\delta i} B_{\beta i \mu} C_{\delta \nu \mu k} E_{\nu k \alpha}$$



(v) Trace of product of six matrices:

$$T = Tr(A_1 A_2 A_3 A_4 A_5 A_6)$$

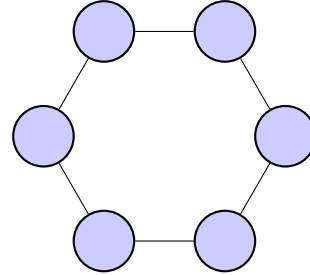


Figure 2.2: Different examples of tensor contractions with their respective tensor diagrams.

From the above examples, it is evident how simplified the tensor contraction becomes with the diagrammatic approach. In fact it is analogous to the Feynman's diagram for tensor network.

It is important to note that the total number of operations that must be done to obtain the final result of a TN contraction depends heavily on the order in which indices in the TN are contracted. Our aim is to make the contraction as efficiently as possible, for example:

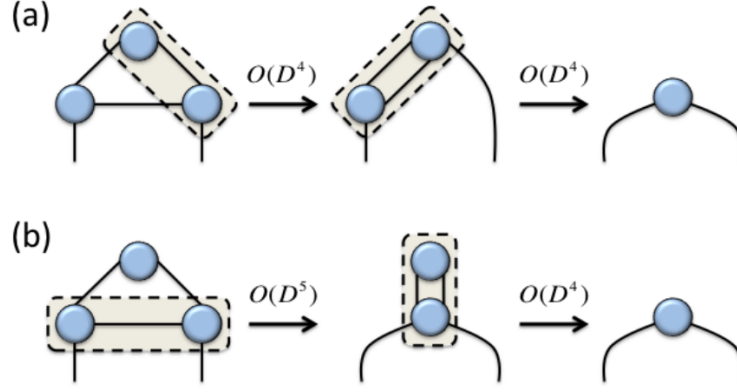


Figure 2.3: Importance of order of index contraction. (The figure is taken from Ref [2])

In the fig.2.3, we can see two cases for the same tensor contraction operation. There are a total of 5 legs here with 3 being connecting indices and 2 being free index. In (a) we see the two contractions performed is of the $O(D^4)$. This is because, while we contract the upper right side, we have to take care of 4 legs of the tensors which are contracted: the contacting leg, remaining three legs associated to them accordingly, and the other tensor is left undisturbed in the entire process to create the new tensor. Whereas in (b) we are contracting the lower two tensors, which requires all 5 legs one to contract and other 4 getting updated to create the new tensor state.

With the above points, we will see how a quantum state can be represented using TN diagrams and decompose the complex tensor coefficients of an N body quantum state into a network of simpler tensor blocks.

2.2.2 Construction of wave-function with small pieces

As discussed in Chapter 1, the efficiency of tensor network decompositions stems from the limited entanglement structure of physically relevant quantum states, allowing a compressed representation using interconnected lower-rank tensors.

Let us see how TN can represent a quantum state. Let us consider a quantum many-body system of N particles. The complete quantum state of the system is described by a wavefunction in the many-body Hilbert space. Without the loss of generality, we can take each particle have p degrees of freedom for the ease of understanding. The wavefunction for this system expanded in the computational basis is given by

$$|\psi\rangle = \sum_{i_1 i_2 \dots i_N=1}^p C_{i_1 i_2 \dots i_N} |i_1 i_2 \dots i_N\rangle \quad (2.6)$$

where each $|i_r\rangle$ is an individual basis of each particle and $r = 1, 2, \dots, N$. The Hilbert space is for this system $\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \dots \otimes \mathcal{H}_N$ where $\mathcal{H}_r$ is the Hilbert space of the r^{th} particles. The coefficients of the basis states forms a tensor C with N indices $i_1, i_2, \dots, i_N$, each taking values upto p, and completely describes the wavefunction. Thus the tensor C has a total of p^N values stored in it. This forms a tensor of rank N with $O(p^N)$ coefficients. With this we can clearly see that the number of parameters that describe the wavefunction $|\psi\rangle$ is exponentially large in the system size. Specifying the values of each one of the coefficients $C_{i_1 i_2 \dots i_N}$ of tensor C is computationally inefficient description of the quantum state of the many body system.

Tensor network shines here. It aims to reduce the complexity in the representation of states like $|\psi\rangle$ by providing an accurate description of the expected entanglement properties of the state. This is achieved by replacing the large tensor C by a TN of “smaller” tensors linked together. The smaller rank tensors are easier to handle. In this representation, a TN typically depends on a polynomial number of parameters making it a computationally efficient description of the quantum state of the many body system.

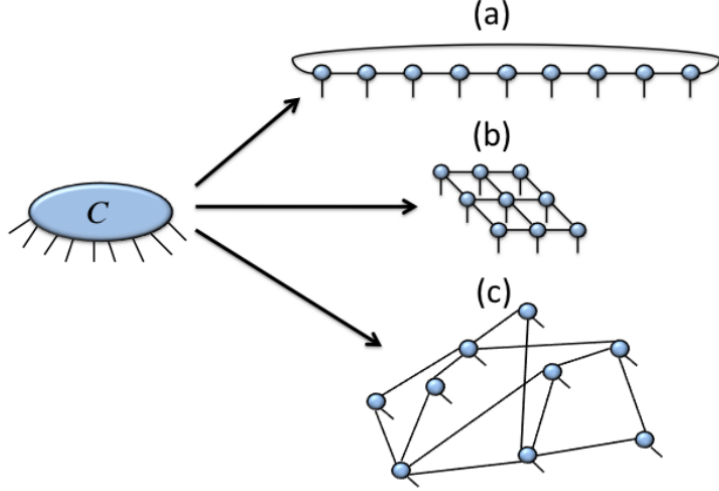


Figure 2.4: Tensor network decomposition of tensor C in terms of (a) MPS with Periodic Boundary Conditions (PBC) (b) PEPS with open boundary condition (c) An arbitrary tensor network (Taken from Ref [2])

2.2.3 Estimation of total number of parameters in a TN

As discussed in the last section, the computational efficiency of a tensor network depends on the total number of independent parameters stored across all constituent tensors. The total number of parameters m_{tot} [2] in the TN is given by:

$$m_{tot} = \sum_{t=1}^{N_{tens}} m(t) \quad (2.7)$$

where $m(t)$ is the number of parameters for tensor t in the TN and N_{tens} is the number of tensors.

For a TN to be practical N_{tens} must be sub exponential in N . That is $N_{tens} = O(poly(N))$ and sometimes this can be even $N_{tens} = O(1)$ which means constant. Now for estimating the number of parameters shall take up the following case. For each tensor t the number of parameter is

$$m(t) = O \left(\prod_{\alpha_t}^{rank(t)} D(\alpha_t) \right) \quad (2.8)$$

where $\alpha_t = 1, 2, \dots, rank(t)$ are indices of the tensor t , $D(\alpha_t)$ are the different possible values of index α_t and $rank(t)$ is the number of indices of the tensor t .

To estimate the upper bound, we take D_t the maximum of all the numbers $D(\alpha_t)$ for a given tensor. So, the maximum value of $m(t)$ is given by:

$$m(t) = O(D_t^{rank(t)}) \quad (2.9)$$

Therefore, the total number of parameters will be

$$m_{tot} = \sum_{t=1}^{N_{tens}} O(D_t^{rank(t)}) = O(poly(N)poly(D)) \quad (2.10)$$

where D is the maximum of D_t over all tensors assuming that the rank of each tensor is bounded by a constant.

Let us discuss an example of a TN by taking a Matrix Product State (MPS) with periodic boundary condition (cf fig2.4(a)). Each tensor site (the circles) has one free leg which is the free index and two legs connecting other tensor sites. Assuming that the open indices in the TN can take up to p values and the rest can take up to D values, we have the number of parameter as $O(NpD^2)$. This can be understood more clearly with fig.2.5

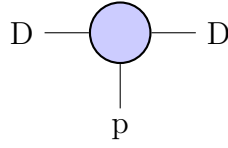


Figure 2.5: A rank-3 tensor representing a single site of a Matrix Product State. The horizontal legs denote the virtual (bond) indices both taking up to D values, while the vertical leg denotes the physical index taking up to p values.

From the fig.2.5 we can see, for a single tensor site we have the number of parameters being $O(pD^2)$. Therefore, in the MPS with periodic boundary condition, we have N such sites being repeated, hence giving the value $O(NpD^2)$. The contraction of the TN yields a tensor of rank N and therefore p^N coefficients. These coefficients are not independent but rather they are obtained from the contraction of a given TN and therefore have a structure.

2.3 Components of the Tensor Network

We have so far seen that the “big” tensor C can be broken down into “smaller” tensors and a network is formed by connecting them. The replacement of tensor C by a TN involves the appearance of **extra degrees of freedom** in the system. They are responsible for coupling different tensor blocks together. These are represented by the connecting indices amongst the tensors in the TN. These extra degrees of freedom are not merely a mathematical artifact, they have their physical significance. They represent the structure of the many-body entanglement in the quantum state $|\psi\rangle$. The number of different values that each one of these indices can take is a quantitative measure of the amount of the quantum correlations in the wave function. These indices are usually called **(virtual) bonds** or **ancillary indices** and their number of possible values are referred to as **bond dimensions**. The concept of bond dimension is central to tensor network methods. It determines the amount of quantum correlations that can be encoded within the network and therefore controls both the expressive power and computational cost of the ansatz. In the next chapter, we introduce the Matrix Product State (MPS), one of the most successful tensor network architectures for one-dimensional quantum systems, and investigate how these virtual bonds naturally arise together with the canonical forms that make MPS particularly suitable for numerical simulations.

In this chapter we have developed the basic mathematical framework underlying tensor networks. Beginning with tensors and their graphical representation, we showed how complicated tensor contractions can be expressed using intuitive diagrammatic notation. We then demonstrated that the coefficient tensor describing a many-body quantum state grows exponentially with the system size, making a direct representation computationally prohibitive. Tensor networks overcome this difficulty by decomposing the large coefficient tensor into a network of interconnected lower-rank tensors, where the virtual bonds encode the entanglement structure of the quantum state.

Chapter 3

Matrix Product State (MPS) and its Canonical Form

Among the various tensor network architectures developed for representing quantum many-body states, the Matrix Product State (MPS) has emerged as one of the most successful ansätze for one-dimensional systems. Besides providing an efficient representation of quantum states with limited entanglement, MPS possesses several mathematical properties that make it particularly well suited for numerical algorithms and ground-state calculations. In this chapter, we develop the formalism of Matrix Product States, discuss their canonical forms, transfer matrices and introduce Matrix Product Operators (MPOs), which provide an analogous representation for operators acting on these states.

3.1 What are MPS?

A Matrix Product State (MPS) is a type of tensor network in which the coefficient tensor of a one-dimensional quantum many-body wavefunction is decomposed into a 1-d chain of interconnected tensors. An MPS consists of one tensor associated with each lattice site of the one-dimensional quantum system. The connecting bond indices that glue the tensors together can take up to D values. The open indices correspond to the physical degrees of freedom of the local Hilbert spaces which can take up to p values.

MPS can be categorized based on its size and boundary conditions. Let us first discuss based on the system size. MPS with finite number of tensor sites are called finite MPS and those forming an infinite chain are called infinite MPS (iMPS). Finite MPS are used to simulate systems with finite number of quantum-many bodies. Whereas iMPS are used to study thermodynamic limit of a system where $N \rightarrow \infty$ and they are very useful for system which show 1d translational invariance. We will discuss more about this in the properties of MPS.

MPS are of two types based on the boundary conditions namely: (a) Open Boundary Condition (OBC) and (b) Periodic Boundary Condition (PBC). In open boundary condition, the TN is composed of rank-3 tensors with rank-2 tensors at the boundaries. In periodic boundary condition, we have similar chain structure of the open boundary condition, with the additional feature that the first and last tensor sites are now linked forming a ring-like structure. They can be understood easily with the following diagram:

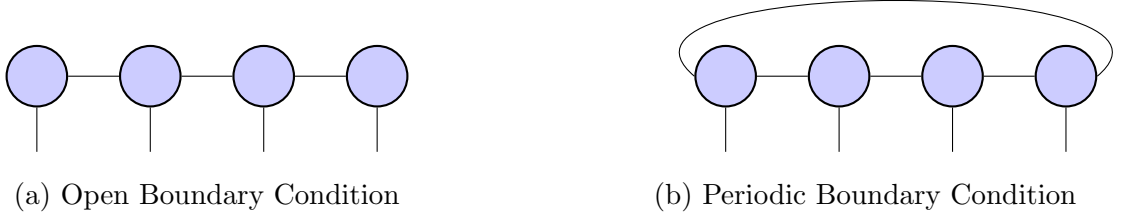


Figure 3.1: Matrix Product States with (a) Open Boundary Condition (OBC) and (b) Periodic Boundary Condition (PBC).

A state $|\psi\rangle$ with open boundary condition[7] is written in an equation as:

$$|\psi\rangle_{obc} = \sum_{\{i\}} A_{i_1}^{[1]} A_{i_2}^{[2]} \dots A_{i_N}^{[N]} |i_1 i_2 \dots i_N\rangle, \quad (3.1)$$

and with periodic boundary condition[7], it is written as:

$$|\psi\rangle_{pbc} = \sum_{\{i\}} \text{tr}(A_{i_1}^{[1]} A_{i_2}^{[2]} \dots A_{i_N}^{[N]}) |i_1 i_2 \dots i_N\rangle \quad (3.2)$$

where $A_{i_1}^{[1]}, \dots, A_{i_N}^{[N]}$ are the matrices representing the tensor sites.

3.2 Canonical Form of MPS

An MPS representation of a quantum state is not unique. Different sets of tensors may represent exactly the same physical wavefunction. Given a quantum state $|\psi\rangle$ in terms of an MPS with open boundary conditions, there is choice of tensors called canonical form of the MPS which is extremely convenient. This freedom of choosing the convenient basis is known as gauge freedom. The canonical form is obtained by exploiting the gauge freedom of an MPS. There are three forms of canonical transformations:

- Left Canonical Form
- Right Canonical Form
- Mixed Canonical Form

For a given MPS with open boundary condition (either for a finite or infinite system), we say it is in its canonical form if for each bond index α , the index corresponds to the labelling of Schmidt vectors in the Schmidt decomposition [2] of $|\psi\rangle$ across that index:

$$|\psi\rangle = \sum_{\alpha=1}^D \lambda_{\alpha} |\phi_{\alpha}^L\rangle \otimes |\phi_{\alpha}^R\rangle \quad (3.3)$$

where λ_{α} is the Schmidt coefficients, $|\phi_{\alpha}^L\rangle$ and $|\phi_{\alpha}^R\rangle$ are the left and right Schmidt basis vectors.

The above picture is the beginning step of constructing an MPS into its canonical form. For this we need to first equip ourselves with **Schmidt decomposition**. It is the key technique used to convert an MPS into its canonical form.

3.2.1 Schmidt Decomposition

Schmidt decomposition expresses any pure bipartite quantum state as a sum of orthogonal product states. It can be seen in the following equation:

$$|\psi\rangle = \sum_{\alpha=1}^D \lambda_{\alpha} |\alpha_A\rangle \otimes |\alpha_B\rangle \quad (3.4)$$

where λ_{α} is the Schmidt coefficients are real numbers and are ordered in decreasing order, i.e., $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D$, $|\alpha_A\rangle$ and $|\alpha_B\rangle$ are the Schmidt basis vectors of system A and B respectively. The Schmidt vectors are orthonormal to each other that is

$$\langle \alpha_A | \alpha'_A \rangle = \langle \alpha_B | \alpha'_B \rangle = \delta_{\alpha\alpha'} \quad (3.5)$$

Another key property of the Schmidt decomposition is that the sum of the squares of the Schmidt coefficients must be equal to 1:

$$\sum_{\alpha=1}^D \lambda_{\alpha}^2 = 1 \quad (3.6)$$

This is to make the norm of $|\psi\rangle$ be unity, i.e., $\langle \psi | \psi \rangle = 1$.

To perform Schmidt decomposition, we need to Singular Value Decomposition (SVD) on the coefficients of basis of vectors of $|\psi\rangle$. To see this in action, let us take a bipartite system with the Hilbert space $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B$. Let $|i_A\rangle$ and $|j_B\rangle$ be the computational basis of $\mathcal{H}_A$ and $\mathcal{H}_B$ respectively. Therefore, the state $|\psi\rangle$ in the Hilbert space $\mathcal{H}$ is represented as:

$$|\psi\rangle = \sum_{i,j} c_{i,j} |i_A\rangle |j_B\rangle \quad (3.7)$$

The coefficients c_{ij} forms a tensor C of rank 2, so it can be represented as a matrix. Now we can perform SVD on C which yields:

$$C = U \Lambda V^{\dagger} \quad (3.8)$$

where U and $V^{\dagger}$ are unitary matrices and Λ is a diagonal matrix such that $\Lambda_{\alpha\alpha} = \lambda_{\alpha}$ and $\Lambda_{\alpha\alpha'} = 0$ for $\alpha \neq \alpha'$

So the state $|\psi\rangle$ is now:

$$\begin{aligned} |\psi\rangle &= \sum_{i,j,\alpha} u_{i\alpha} \Lambda_{\alpha\alpha} v_{\alpha j}^* |i_A\rangle |j_B\rangle \\ &= \sum_{\alpha} \lambda_{\alpha} \left(\sum_i u_{i\alpha} |i_A\rangle \right) \left(\sum_j v_{\alpha j}^* |j_B\rangle \right) \\ &= \sum_{\alpha=1}^D \lambda_{\alpha} |\alpha_A\rangle \otimes |\alpha_B\rangle \end{aligned} \quad (3.9)$$

where u_{ij} and v_{ij} are the elements of U and V matrices respectively.

The Schmidt decomposition provides a natural basis for bipartite quantum systems and plays a central role in tensor network methods. By repeatedly applying Schmidt decompositions across successive bipartitions of a one-dimensional lattice, the many-body wavefunction can be transformed into the canonical form of an MPS. This we will see in the next section.

3.2.2 Construction of the canonical form

Before deriving the canonical form, let us look what it looks like. For a finite system of N sites, the decomposition for the coefficient of the wavefunction has the form:

$$C_{i_1 i_2 \dots i_N} = \Gamma_{\alpha_1}^{[1] i_1} \lambda_{\alpha_1}^{[1]} \Gamma_{\alpha_1 \alpha_2}^{[2] i_2} \lambda_{\alpha_2}^{[2]} \Gamma_{\alpha_2 \alpha_3}^{[3] i_3} \lambda_{\alpha_3}^{[3]} \dots \lambda_{\alpha_{N-1}}^{[N-1]} \Gamma_{\alpha_N}^{[N] i_N} \quad (3.10)$$

where Γ tensors corresponds to changes of basis between the different Schmidt basis and the computational basis, and the vectors λ corresponds to the Schmidt coefficients.

For an infinite MPS with one-site translation invariance, the canonical form corresponds to having just one tensor Γ and one vector λ describing the whole state.

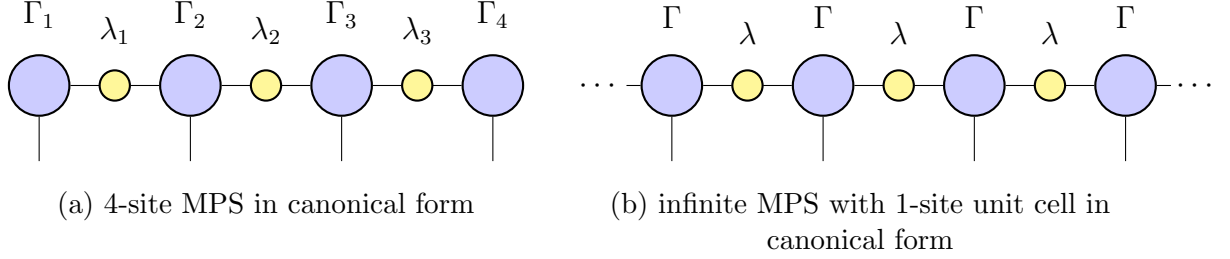


Figure 3.2: Canonical forms of (a) 4-site MPS and (b) iMPS with 1-site unit cell

Derivation:

Let us take the coefficient of the wavefunction $C_{i_1 i_2 \dots i_N}$, with each i_r taking p values for $r = 1, 2, \dots, N$. Let us take the bipartition of $\mathcal{H}$ as $\mathcal{H}_1 \otimes (\mathcal{H}_2 \otimes \dots \otimes \mathcal{H}_N)$. We can now express the tensor C as a matrix although it is still a tensor of rank N in the following manner:

$$C_{i_1 i_2 \dots i_N} \equiv C_{i_1, (i_2 i_3 \dots i_N)} = \begin{pmatrix} C_{1,(1\dots 1)} & C_{1,(1\dots 2)} & \dots & C_{1,(p\dots p)} \\ C_{2,(1\dots 1)} & C_{2,(1\dots 2)} & \dots & C_{2,(p\dots p)} \\ \vdots & \vdots & \ddots & \vdots \\ C_{p,(1\dots 1)} & C_{p,(1\dots 2)} & \dots & C_{p,(p\dots p)} \end{pmatrix}. \quad (3.11)$$

Here, the tensor C is reshaped into a matrix by treating i_1 as the row index, while the remaining indices $(i_2, i_3, \dots, i_N)$ are grouped together into a single composite column index. No information is lost in this reshaping; only the way the tensor is indexed is changed.

We can now perform Schmidt decomposition between the site 1 and the remaining $N-1$ sites. This decomposes the many-body wavefunction into orthonormal basis states associated with the two subsystems,

$$|\Psi\rangle = \sum_{\alpha_1=1}^{\min(p,D)} \lambda_{\alpha_1}^{[1]} |\tau_{\alpha_1}^{[1]}\rangle \otimes |\tau_{\alpha_1}^{[2,\dots,N]}\rangle. \quad (3.12)$$

where $\lambda_{\alpha_1}^{[1]}$ are the Schmidt coefficients for the bipartition, and the states $|\tau_{\alpha_1}^{[1]}\rangle$ and $|\tau_{\alpha_1}^{[2,\dots,N]}\rangle$ are the left and right Schmidt basis vectors.

Since the left subsystem consists of only a single lattice site, its Schmidt basis can be expanded in terms of the local basis state $|i_1\rangle$ as

$$|\tau_{\alpha_1}^{[1]}\rangle = \sum_{i_1=1}^p \Gamma_{\alpha_1}^{[1] i_1} |i_1\rangle, \quad (3.13)$$

where $\Gamma_{\alpha_1}^{[1]i_1}$ denotes the change-of-basis coefficients from the local basis to the Schmidt basis. Substituting this expansion into the Schmidt decomposition gives

$$\begin{aligned} |\Psi\rangle &= \sum_{\alpha_1}^{\min(p,D)} \lambda_{\alpha_1}^{[1]} \left(\sum_{i_1}^p \Gamma_{\alpha_1}^{[1]i_1} |i_1\rangle \right) \otimes |\tau_{\alpha_1}^{[2,\dots,N]}\rangle \\ &= \sum_{i_1=1}^p \sum_{\alpha_1=1}^{\min(p,D)} \Gamma_{\alpha_1}^{[1]i_1} \lambda_{\alpha_1}^{[1]} |i_1\rangle \otimes |\tau_{\alpha_1}^{[2,\dots,N]}\rangle. \end{aligned} \quad (3.14)$$

The right Schmidt vector $|\tau_{\alpha_1}^{[2,\dots,N]}\rangle$ still represents a quantum state of the remaining $N - 1$ sites. It can therefore be expanded in terms of the local basis of the second site as

$$|\tau_{\alpha_1}^{[2,\dots,N]}\rangle = \sum_{i_2=1}^p |i_2\rangle \otimes |\omega_{\alpha_1 i_2}^{[3,\dots,N]}\rangle. \quad (3.15)$$

The states $|\omega_{\alpha_1 i_2}^{[3,\dots,N]}\rangle$ are generally not orthonormal. We therefore perform another Schmidt decomposition between the subsystem consisting of the first two sites and the remaining sites,

$$|\omega_{\alpha_1 i_2}^{[3,\dots,N]}\rangle = \sum_{\alpha_2=1}^{\min(p^2,D)} \Gamma_{\alpha_1 \alpha_2}^{[2]i_2} \lambda_{\alpha_2}^{[2]} |\tau_{\alpha_2}^{[3,\dots,N]}\rangle. \quad (3.16)$$

Since the left subsystem now consists of two physical sites, its Hilbert space has dimension at most p^2 . Therefore the Schmidt rank is bounded by $\min(p^2, D)$. Substituting this expression into the previous equation gives

$$|\Psi\rangle = \sum_{i_1, i_2=1}^p \sum_{\alpha_1}^{\min(p,D)} \sum_{\alpha_2}^{\min(p^2,D)} \left(\Gamma_{\alpha_1}^{[1]i_1} \lambda_{\alpha_1}^{[1]} \Gamma_{\alpha_1 \alpha_2}^{[2]i_2} \lambda_{\alpha_2}^{[2]} \right) |i_1\rangle \otimes |i_2\rangle \otimes |\tau_{\alpha_2}^{[3,\dots,N]}\rangle.$$

The above procedure can be repeated successively for every lattice site. After performing Schmidt decompositions across all bipartitions of the chain, the many-body wavefunction assumes the Matrix Product State representation

$$\begin{aligned} |\Psi\rangle &= \sum_{\{i\}} \sum_{\{\alpha\}} \Gamma_{\alpha_1}^{[1]i_1} \lambda_{\alpha_1}^{[1]} \Gamma_{\alpha_1 \alpha_2}^{[2]i_2} \lambda_{\alpha_2}^{[2]} \dots \Gamma_{\alpha_{N-2} \alpha_{N-1}}^{[N-1]i_{N-1}} \lambda_{\alpha_{N-1}}^{[N-1]} \Gamma_{\alpha_{N-1}}^{[N]i_N} \\ &\quad \times |i_1\rangle \otimes |i_2\rangle \otimes \dots \otimes |i_N\rangle. \end{aligned} \quad (3.17)$$

where the summation over every physical index i_k and bond index α_k is implied. The entire process can be visualized as shown in the fig.3.3 and fig.3.4.

The successive Schmidt decompositions performed across each bond naturally generate the tensors Γ and the Schmidt coefficient vectors λ . Together, these constitute the canonical Matrix Product State representation of the many-body wavefunction. The canonical form makes the entanglement structure of the state explicit and serves as the foundation for many tensor-network algorithms such as DMRG, TEBD and VUMPS.

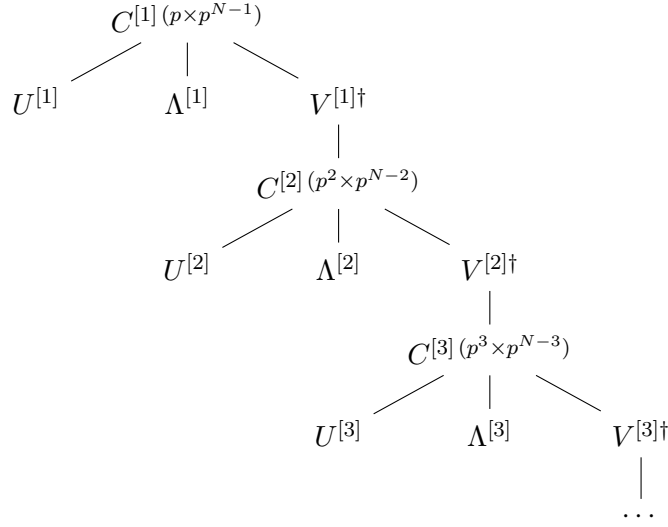


Figure 3.3: Successive singular value decompositions used in constructing the MPS representation. At each step the coefficient tensor is reshaped into a matrix and decomposed as $C = U\Sigma V^\dagger$. The remaining matrix is recursively decomposed until the complete MPS is obtained.

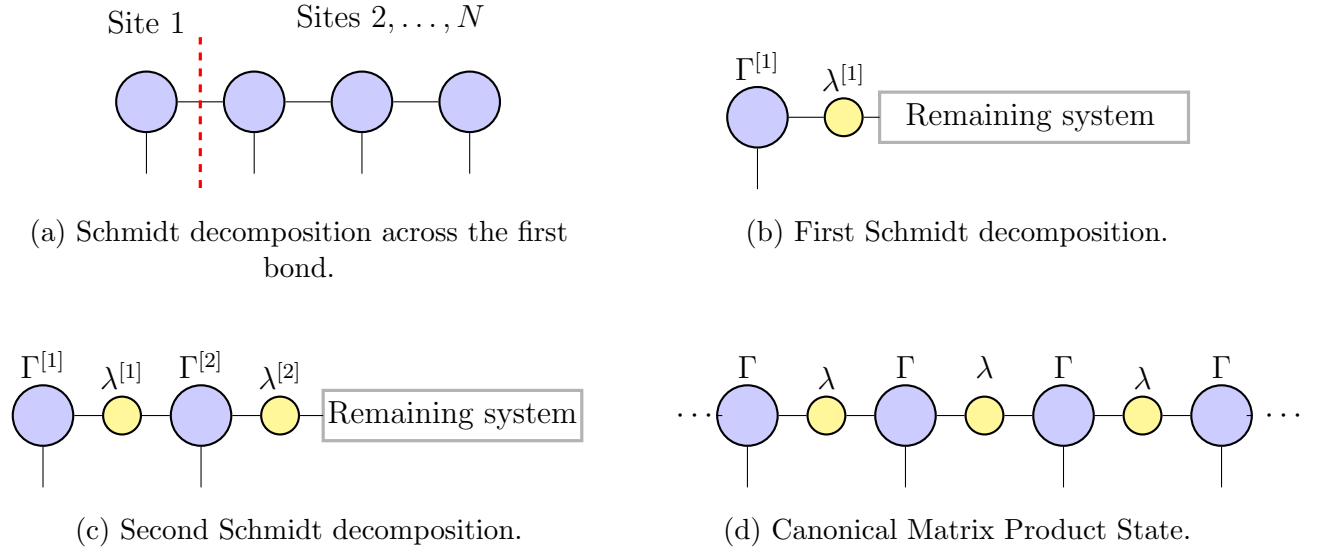


Figure 3.4: Successive construction of the canonical Matrix Product State using repeated Schmidt decompositions. (a) The first Schmidt decomposition partitions the system into site 1 and the remaining sites. (b) The first decomposition extracts the tensor $\Gamma^{[1]}$ and the Schmidt coefficients $\lambda^{[1]}$. (c) Repeating the procedure on the remaining subsystem produces the next Γ - λ pair. (d) Continuing recursively over the entire chain yields the canonical Matrix Product State.

3.2.3 Canonical Forms of finite MPS

The canonical form of a Matrix Product State (MPS)[2][7] is obtained by exploiting the gauge freedom present in the tensor network representation. Different gauge choices lead to different canonical forms, each of which is useful for simplifying tensor contractions and numerical algorithms. The three commonly used canonical forms are the left canonical form, the right canonical form and the mixed canonical form.

Left Canonical Form

An MPS is said to be in the left canonical form if every tensor satisfies the left orthonormality condition

$$\sum_s (A^s)^\dagger A^s = I. \quad (3.18)$$

This condition implies that the effective basis states constructed from the tensors to the left of a given bond are orthonormal. Consequently, contractions involving the left part of the network simplify considerably, making this representation useful in numerical computations.

Right Canonical Form

Similarly, an MPS is in the right canonical form if each tensor satisfies

$$\sum_s A^s (A^s)^\dagger = I. \quad (3.19)$$

In this case, the effective basis states to the right of every bond form an orthonormal basis. The right canonical form is therefore the counterpart of the left canonical form and provides analogous simplifications when contracting tensors from the right.

Mixed Canonical Form

The most useful representation in practical tensor network algorithms is the mixed canonical form. Here, the tensors to the left of a chosen site satisfy the left canonical condition, while the tensors to the right satisfy the right canonical condition. The selected tensor, known as the *orthogonality centre*, connects the two normalized parts of the chain and contains the local information of the quantum state.

This representation naturally emerges from successive Schmidt decompositions and is widely employed in algorithms such as Density Matrix Renormalization Group (DMRG), Time-Evolving Block Decimation (TEBD) etc.

3.2.4 Canonical Form of an Infinite MPS

For an infinite Matrix Product State (iMPS), the canonical form is obtained by orthonormalizing the virtual bond indices. In the canonical form, every bond index corresponds to a pair of orthonormal Schmidt bases on the left and right of the bond. Consequently, the diagonal matrix of Schmidt coefficients is associated with every virtual bond.

For a translationally invariant iMPS described by a single tensor A , the canonical form can be constructed through the following procedure.

Step 1: Dominant Eigenmatrices We first define the transfer operator

$$T(M) = \sum_s A^s M A^{s\dagger}, \quad (3.20)$$

where M is an arbitrary square matrix, $A^s = \Gamma^s \lambda$ and $A^s = (\Gamma^s)^* \lambda$. The dominant right and left eigenmatrices V_R and V_L satisfy

$$T(V_R) = \eta V_R, \quad (3.21)$$

and

$$T^*(V_L) = \eta V_L, \quad (3.22)$$

where η is the dominant eigenvalue of the transfer operator.

The eigenmatrices V_R and V_L are Hermitian and positive definite, and may therefore be factorized as

$$V_R = X X^\dagger, \quad V_L = Y^\dagger Y, \quad (3.23)$$

where X and Y are the gauges for the canonicalization of iMPS. Eq(3.23) is known as Cholesky Decomposition in Linear Algebra.

Step 2: Gauge Transformation The identity operators

$$I = X^{-1} X, \quad I = (Y^T)^{-1} Y^T, \quad (3.24)$$

are inserted on every virtual bond of the MPS. This corresponds to a gauge transformation that leaves the physical quantum state unchanged while changing the tensor representation. The matrix product

$$Y^T \lambda X \quad (3.25)$$

is then decomposed using the singular value decomposition,

$$Y^T \lambda X = U \lambda' V, \quad (3.26)$$

where U and V are unitary matrices and λ' contains the new Schmidt coefficients.

Step 3: Construction of the Canonical Tensor Finally, the remaining matrices are absorbed into a new tensor

$$\Gamma' = V X^{-1} \Gamma (Y^T)^{-1} U, \quad (3.27)$$

so that the infinite MPS is expressed entirely in terms of the tensor Γ' and the diagonal Schmidt matrix λ' .

The resulting representation is in canonical form by construction, with every virtual bond corresponding to orthonormal Schmidt bases. This form is widely used in tensor network algorithms owing to its numerical stability and the simplification it provides for tensor contractions.

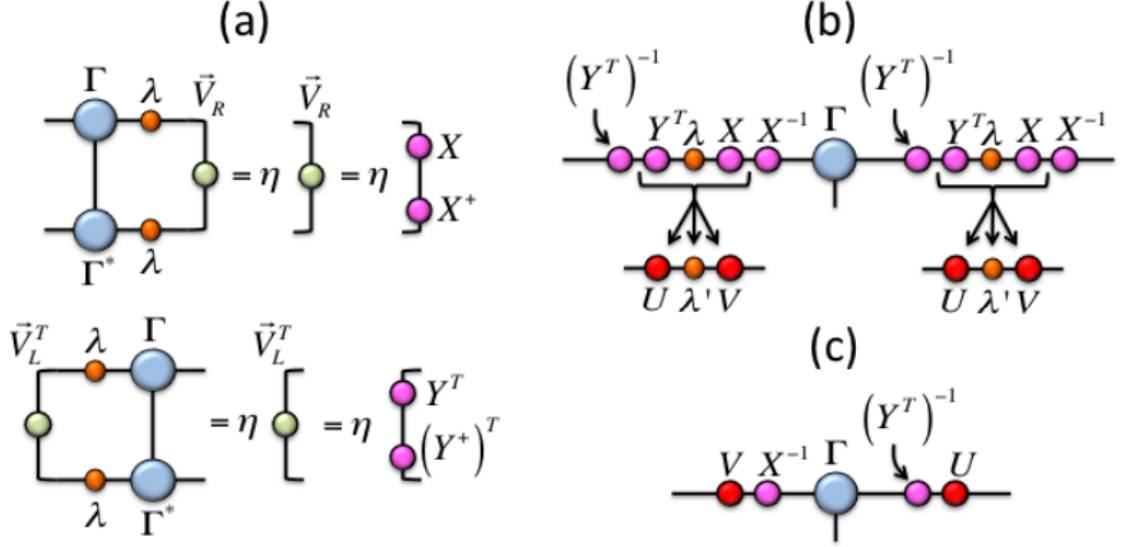


Figure 3.5: Canonicalization procedure for an infinite Matrix Product State. (a) The dominant right eigenmatrix of the transfer operator is computed and factorized as $V_R = XX^\dagger$. Similarly, the dominant left eigenmatrix is obtained as $V_L = Y^\dagger Y$. (b) Appropriate gauge transformations are inserted on the virtual bonds and the resulting matrix product is decomposed using singular value decomposition. (c) The remaining matrices are absorbed into a new tensor Γ' , yielding the canonical form of the infinite MPS. (taken from Ref [2])

3.2.5 Properties of the Canonical Form of an MPS

The canonical form of an MPS possesses several important properties that greatly simplify both the theoretical analysis and numerical implementation of tensor network algorithms. Some of the key properties are listed below.

- The eigenvalues of the reduced density matrix corresponding to any bipartition of the system are simply the squares of the Schmidt coefficients,

$$\mu_\alpha = \lambda_\alpha^2.$$

Consequently, the Schmidt coefficients provide direct access to the entanglement spectrum and enable the straightforward evaluation of entanglement measures such as the von Neumann entropy

- The canonical form considerably simplifies the evaluation of expectation values of local observables. With the orthonormality of the left and right Schmidt bases, many tensor contractions reduce to identity operators, thereby significantly lowering the computational complexity.
- The canonical representation provides an optimal truncation of the bond dimension. By retaining only the largest Schmidt coefficients, one obtains the best approximation to the original quantum state in the least-squares sense.
- The Density Matrix Renormalization Group (DMRG) and Time-Evolving Block Decimation (TEBD) are formulated using the canonical representation because of its numerical stability and computational efficiency.

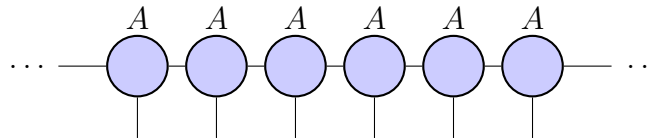
3.3 Correlation in MPS

One of the most remarkable properties of Matrix Product States is that their correlation functions[2] decay exponentially with the separation between two local operators. Consequently, every MPS possesses a finite correlation length. This is in contrast to critical or scale-invariant systems, where the correlation length diverges and the correlations typically decay algebraically. Therefore, a finite bond dimension MPS provides an efficient description of gapped one-dimensional quantum systems, while an exact description of critical systems requires the bond dimension to grow.

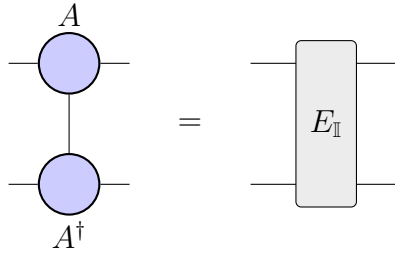
To understand the origin of this exponential decay, let us take an infinite translationally invariant MPS represented by a single tensor A . Since the same tensor is repeated over every lattice site, the entire network can be conveniently expressed in terms of a *transfer matrix*.

$$E = \sum_s A^s \otimes (A^\dagger)^s, \quad (3.28)$$

In operator form we have eq(3.20). This could be understood with following diagram:



(a) iMPS of 1-site unit cell



(b) Transfer Matrix of the iMPS

Figure 3.6: The transfer matrix $E_{\parallel}$ is obtained by contracting a tensor with its complex conjugate over the physical index. Repeated applications of the transfer matrix naturally arise in an infinite translationally invariant MPS.

For an infinite translationally invariant MPS, contracting one lattice site with its conjugate repeatedly is equivalent to repeatedly applying the transfer operator. Consequently, all expectation values and correlation functions can be expressed entirely in terms of successive powers of the transfer operator.

The two-body correlation function between two local operators O_i and O'_{i+r} which are separated by a distance r (in the units of site position) is defined as

$$C(r) = \langle O_i O'_{i+r} \rangle - \langle O_i \rangle \langle O'_{i+r} \rangle. \quad (3.29)$$

Diagrammatically:

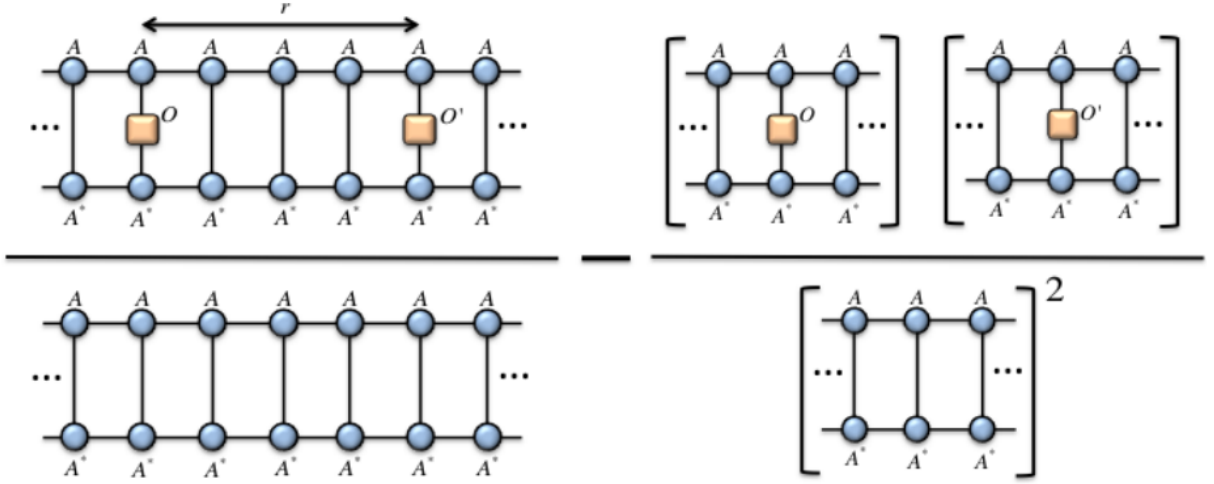


Figure 3.7: The two-body correlator $C(r)$ represented by TN diagrams (taken from Ref [2])

Let the eigenvalues of the transfer matrix be ordered according to

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \quad (3.30)$$

Since repeated powers of the transfer matrix amplify only the dominant eigenvalues, and assuming λ_2 has ω degeneracy, we obtain

$$E_{\mathbb{I}}^r \sim \lambda_1^r \left(|R_1\rangle \langle L_1| + \left(\frac{\lambda_2}{\lambda_1} \right)^r \sum_{\mu=2}^{\omega+1} |R_\mu\rangle \langle L_\mu| + \dots \right). \quad (3.31)$$

Defining the matrices $E_{\mathbb{O}}$ and $E'_{\mathbb{O}}$ as:

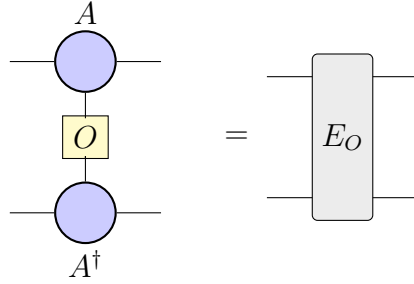


Figure 3.8: $E_{\mathbb{O}}$ is the transfer matrix with local operator O . Same diagram will be for the operator O' .

Now, using the eq(3.31) we can estimate

$$\begin{aligned} \langle O_i O'_{i+r} \rangle &\sim \langle L_1 | O_i O'_{i+r} | R_1 \rangle \\ &= \frac{1}{\lambda_1^2} \langle L_1 | R_1 \rangle \langle L_1 | E_{\mathbb{O}} | R_1 \rangle \langle L_1 | E'_{\mathbb{O}} | R_1 \rangle \\ &\quad + \frac{1}{\lambda_1^2} \left(\frac{\lambda_2}{\lambda_1} \right)^{r-1} \sum_{\mu=2}^{\omega+1} \langle L_1 | E_{\mathbb{O}} | R_\mu \rangle \langle L_\mu | E'_{\mathbb{O}} | R_1 \rangle \langle L_1 | R_1 \rangle \\ &= \frac{1}{\lambda_1^2} \langle L_1 | E_{\mathbb{O}} | R_1 \rangle \langle L_1 | E'_{\mathbb{O}} | R_1 \rangle + \left(\frac{\lambda_2}{\lambda_1} \right)^{r-1} \frac{1}{\lambda_1^2} \sum_{\mu=2}^{\omega+1} \langle L_1 | E_{\mathbb{O}} | R_\mu \rangle \langle L_\mu | E'_{\mathbb{O}} | R_1 \rangle \end{aligned} \quad (3.32)$$

It is factor $1/\lambda_1^2$ can be understood intuitively from the fig.3.7. There two sites occupied by the operators O and O' at i and $i+r$. So by eq.(3.31), we have two λ_1 factor in the numerator of the first term. Similarly, we have for the second term missing one term but since being a product we got the same factor.

In eq.(3.32), the first term is nothing but $\langle O_i \rangle \langle O'_{i+r} \rangle$. Therefore, $C(r)$ is given for large r by

$$\begin{aligned} C(r) &\sim \left(\frac{\lambda_2}{\lambda_1}\right)^{r-1} \sum_{\mu=2}^{\omega+1} \langle L_1 | E_{\mathbb{O}} | R_{\mu} \rangle \langle L_{\mu} | E'_{\mathbb{O}} | R_1 \rangle \\ &\sim a \left(\frac{\lambda_2}{\lambda_1}\right)^r \\ &\sim f(r) a e^{-r/\xi} \end{aligned} \quad (3.33)$$

where $a = g(\omega)$ is constant, $f(r) = \pm 1$ is a site-dependent phase if O and O' are hermitian. Since $|\lambda_2| < |\lambda_1|$, the connected correlation function decays exponentially with the separation distance, where the correlation length is

$$\xi = -\frac{1}{\ln |\lambda_2/\lambda_1|}. \quad (3.34)$$

The existence of a finite correlation length is a characteristic feature of Matrix Product States with finite bond dimension. This explains why MPS provides an excellent description of non-critical one-dimensional quantum systems obeying the area law of entanglement, whereas critical systems require increasingly larger bond dimensions to accurately capture their long-range correlations.

3.4 Exact Calculation of the expectation values

The exact calculations of the scalar product between two MPS for N sites can always be done exactly in a time $O(NpD^3)$. This can be understood with the following diagram:

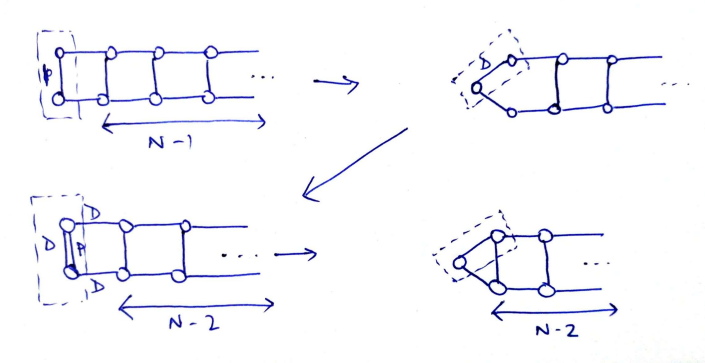


Figure 3.9: Successive contraction of an infinite Matrix Product State. Each local contraction scales as $\mathcal{O}(pD^3)$ since one physical index of dimension p and three virtual indices of dimension D are involved. Repeating this contraction over N sites yields an overall computational complexity of $\mathcal{O}(NpD^3)$.

For an infinite system, the calculation can be done in $\mathcal{O}(pD^3)$ using similar techniques used in the calculation of the two point correlator $C(r)$. By finding the dominant eigenvalue and dominant left/right eigenvectors of the transfer matrix $E_m \mathbb{I}$. In general, expectation values of local observables such as correlation function, and local order parameter, can also be computed using the same kind of tensor manipulations.

For periodic boundary condition, the expectation value is calculated as :

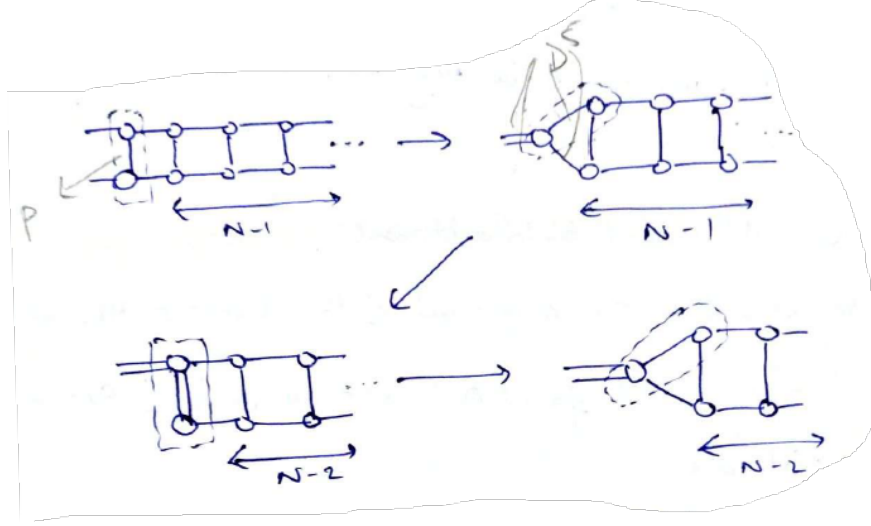


Figure 3.10: fig:Expectation value evaluation of a MPS having PBC

The calculation is done in $\mathcal{O}(NpD^5)$ time. Unlike open boundary conditions, periodic boundary conditions do not permit complete cancellation of the transfer matrices through the canonical form, leading to a significantly higher computational cost.

3.5 Fundamental Properties of MPS

Having established the canonical representation of Matrix Product States and discussed the efficient evaluation of correlation functions and expectation values, we conclude this chapter by highlighting some fundamental properties[2] of MPS. These properties explain why MPS provide an efficient description of one-dimensional quantum many-body systems and motivate their widespread use in modern tensor network algorithms.

3.5.1 Translational Invariance and the Thermodynamic Limit

In principle, all tensors in a finite-size MPS could be different. MPS itself is not translational invariant (TI)[2]. However, it is possible to impose TI and take the thermodynamic limit of the MPS by choosing some fundamental unit cell of tensors that is repeated over the 1d lattice, infinitely many times. This is quite similar to how we study crystals in solid state physics, by defining lattice and unit cells. Let us consider a few examples.

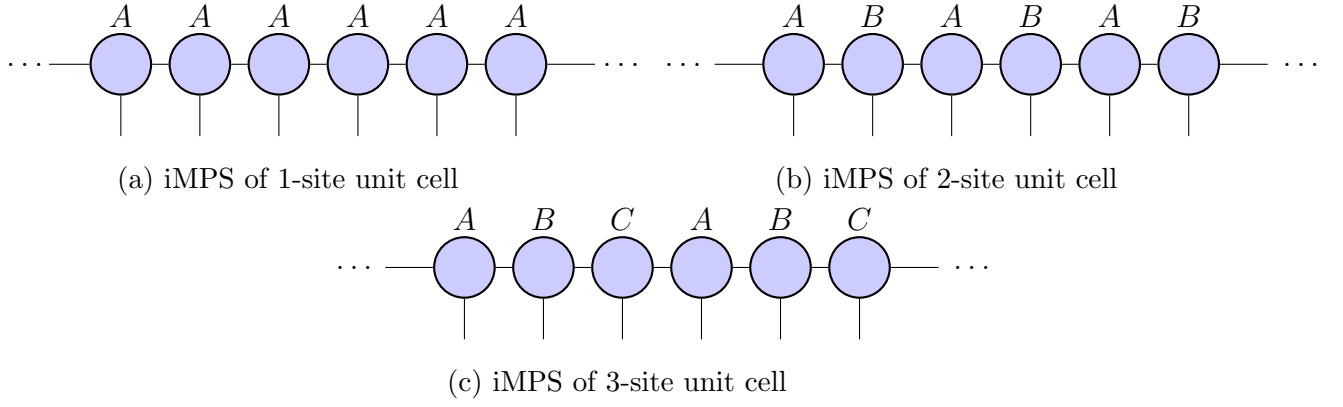


Figure 3.11: Different examples of translational invariance in infinite MPS.

3.5.2 Representational Power of MPS

MPS can represent any quantum state of the many-body Hilbert space just by increasing sufficiently increasing the value of the bond dimension D [2]. To cover all the states in the Hilbert space D needs to exponentially large in the system size. However, it is known that the low energy states of the gapped local Hamiltonians in 1d can be efficiently approximated with almost arbitrary accuracy by an MPS with a finite value of D . For 1d critical systems, D tends to diverge polynomially in the size of the system. This makes MPS-based methods of 1d system such as DMRG accurate.

3.5.3 One-dimensional area-law

MPS satisfy the area-law scaling[2] of the entanglement entropy adapted to 1d systems. The entanglement entropy of a block of sites is bounded by a constant, more precisely $S(L) = -\text{tr}(\rho_L \log \rho_L) = O(\log(D))$ with ρ_L being the reduced density matrix of the block. Exact behaviour that is usually observed in ground states of the gapped 1d local Hamiltonians for large size L of the block:

$$S(L) \sim \text{constant for } L \gg 1$$

Thus, unlike generic quantum states whose entanglement entropy grows extensively with system size, MPS naturally satisfy the one-dimensional area law. Consequently, they provide an efficient variational ansatz for the ground states of gapped local Hamiltonians.

3.6 Matrix Product Operator

We have seen how to a quantum state can be represented in the form of MPS. Similar, to this we can represent the operators in the form of a Tensor Network similar to MPS which is called Matrix Product Operator (MPO)[2][3]. An operator can be expressed as the superposition of the outer products of the basis states. Let us consider an operator $\hat{U}$ acting on some N many-body Hilbert space $\mathcal{H}$ of dimension say d with a basis set $\{|i_1, i_2, \dots, i_N\rangle\}$ which can be represented as

$$\hat{U} = \sum_{\{i\}, \{j\}} u_{i_1 \dots i_N j_1 \dots j_N} |i_1, i_2, \dots, i_N\rangle \langle j_1, j_2, \dots, j_N| \quad (3.35)$$

where u_{ij} are the coefficients of the matrix element $|i\rangle\langle j|$. So the U forms a tensor of rank d^2 .

We can see it shares the similar mathematical structure as the quantum state. Therefore, we can use the same method used for MPS to compress the complexity of the operator and represent it as a chain of local operators which are illustrated below:

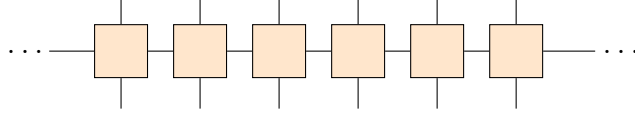


Figure 3.12: Representation of an operator as a Matrix Product Operator (MPO). Each square represents a local tensor of the operator, while the horizontal bonds denote the virtual indices connecting neighboring tensors but there are now two vertical legs as open indices unlike MPS.

Matrix Product Operators provide an efficient tensor network representation of many-body operators in the same way that MPS represent quantum states. Consequently, expectation values reduce to contractions between an MPS, an MPO and the conjugate MPS.

Concluding the discussion on MPS

This chapter established the mathematical framework of Matrix Product States by deriving their canonical representation through successive Schmidt decompositions and discussing the canonical forms for both finite and infinite systems. The transfer matrix formalism was then introduced to explain the exponential decay of correlations and the efficient evaluation of expectation values. Finally, several fundamental properties of MPS were discussed, illustrating why they constitute one of the most successful tensor network ansätze for one-dimensional quantum many-body systems.

Having developed the theoretical foundation of Matrix Product States, we now proceed to the algorithms that are used to approximate the ground states.

Chapter 4

Finding Ground States using MPS

In the previous chapter, we discussed the representation of Matrix Product States (MPS), their canonical forms, and the fundamental properties that make them an efficient description of one-dimensional quantum many-body systems. We now turn to the central question: given a Hamiltonian, how can one determine the MPS that best approximates its ground state?

The strength of tensor network methods lies in their ability to efficiently explore the physically relevant corner of the exponentially large Hilbert space. In particular, MPS provide an excellent variational ansatz for the ground states of one-dimensional gapped local Hamiltonians, making them one of the most successful numerical techniques in quantum many-body physics.

Two of the most widely used approaches for determining the ground state within the MPS framework are

- (i) Variational optimization, and
- (ii) Imaginary time evolution.

Both methods exploit the MPS representation to efficiently approximate the ground state while avoiding the exponential complexity associated with the full Hilbert space. The following sections discuss the principles underlying these approaches and the algorithms derived from them.

4.1 Variational Optimization

Given a Hamiltonian H , the variational principle[2] states that for a given quantum state $|\psi\rangle$ it will always be the case that

$$\frac{\langle\psi|H|\psi\rangle}{\langle\psi|\psi\rangle} \geq E_0, \quad (4.1)$$

where E_0 is the lowest eigenvalue of H (i.e, the ground state energy).

The ground state of a Hamiltonian minimizes the expectation value of the energy. Restricting the search space to Matrix Product States converts the ground-state problem into a variational optimization over the tensor elements. Therefore, for $|\psi\rangle$ if we have MPS with a fixed bond dimension D , we can always approach the ground state energy by minimizing the expectation value:

$$\min (\langle\psi|H|\psi\rangle - \lambda \langle\psi|\psi\rangle) \quad (4.2)$$

where λ is a Lagrange multiplier that introduces the constraint that $|\psi\rangle$ must have norm one.

Ideally, the above normalization should be done simultaneously over the coefficients of all the tensors for all sites of the MPS. However, this is normally quite difficult to implement and not particularly efficient.

To overcome this hurdle, the strategy is to proceed with tensor by tensor. This is to minimize with respect to one tensor while keeping the others fixed, then move to another tensor and repeat the minimization, and so on. In practice one sweeps over all the tensors several times until the desired convergence is attained.

The variational algorithm is described below:

- **Step 1:** Fix all the tensors in the MPS except one of them which we call A . The coefficients of A are now our variational parameters.
- **Step 2:** The minimization is then written as

$$\min_A (\langle \psi | H | \psi \rangle - \lambda \langle \psi | \psi \rangle) = \min_A (\vec{A}^\dagger H_{eff} \vec{A} - \lambda \vec{A}^\dagger \mathcal{N} \vec{A}) \quad (4.3)$$

Here, $\vec{A}$ is the tensor A but with all of its coefficients arranged as a vector, H_{eff} is an effective Hamiltonian and $\mathcal{N}$ is a normalization matrix.

In practice, H_{eff} and $\mathcal{N}$ corresponds respectively to the TNs for $\langle \psi | H | \psi \rangle$ and $\langle \psi | \psi \rangle$ but without tensors A and $A^\dagger$. These are the environment tensors of A and $A^\dagger$ in the two TNs, but written in matrix form.

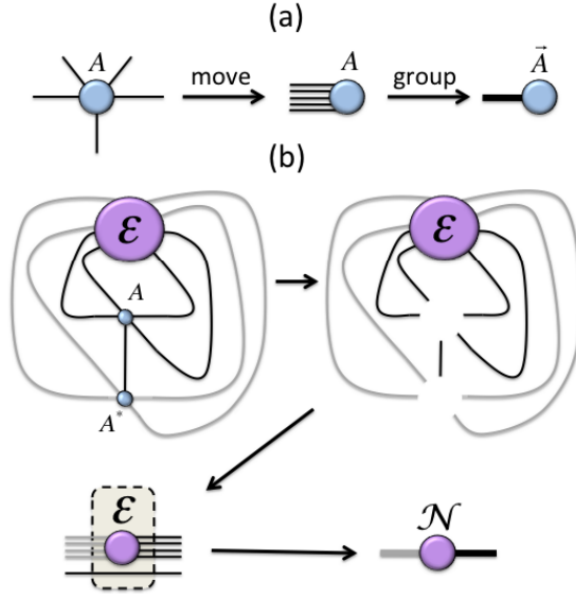


Figure 4.1: (a) Transformation of a tensor into a vector (b) construction of the normalization matrix (taken from Ref [2])

- **Step 3:** The minimization can be done as using Wirtinger derivatives[8]:

$$\frac{\partial}{\partial A^\dagger} (\vec{A}^\dagger H_{eff} \vec{A} - \lambda \vec{A}^\dagger \mathcal{N} \vec{A}) = 0 \quad (4.4)$$

which leads to the generalized eigenvalue equation:

$$H_{eff} \vec{A} - \lambda \mathcal{N} \vec{A} = 0 \quad (4.5)$$

Once H_{eff} and $\mathcal{N}$ are known, this generalized eigenvalue problem can be solved numerically by standard linear algebra package like Krylov algorithm that utilizes Lanczos method. In practice, they can be computed efficiently and exactly for Matrix Product States.

4.1.1 Density Matrix Renormalization Algorithm (DMRG)

This method is the foundation for Density Matrix Renormalization Algorithm (DMRG). It is one of the most effective algorithms for open boundary condition (obc) in MPS. It can be extended to periodic boundary condition (pbc) but the complexity increases compared to obc. Additionally, the variational method is also successful for infinite MPS, finite PEPS, Tree Tensor Network (TTN), Multi-Scale Entanglement Renormalization *Ansatz* (MERA) etc. The optimization is carried out sequentially by updating one tensor at a time while keeping all others fixed. Once a tensor has been optimized, the orthogonality center is shifted to the neighboring site and the procedure is repeated. Successive left-to-right and right-to-left sweeps are performed until the energy converges.

4.2 Imaginary Time Evolution

In quantum mechanics, the time evolution[9] of a state is obtained by applying the propagator $\hat{U}(t, t_0)$ on an initial state $|\psi(t_0)\rangle$ as:

$$|\Psi(t)\rangle = \hat{U}(t, t_0) |\Psi(t_0)\rangle \quad (4.6)$$

By Schrödinger's time evolution equation, we have

$$i\hbar \frac{\partial}{\partial t} |\Psi(t)\rangle = H |\Psi(t)\rangle \quad (4.7)$$

Substituting eqn.(4.6) in eqn.(4.7), we obtain

$$\frac{\partial}{\partial t} \hat{U}(t, t_0) = -\frac{i}{\hbar} H \hat{U}(t, t_0) \quad (4.8)$$

This differential equation can be integrated directly if the Hamiltonian is time independent, we obtain the form of the propagator as:

$$\hat{U}(t, t_0) = \exp\left(-\frac{i}{\hbar} H(t - t_0)\right) \quad (4.9)$$

Substituting $t_0 = 0$, we get the form:

$$\hat{U}(t) = \exp\left(-\frac{i}{\hbar} Ht\right) \quad (4.10)$$

Therefore, the $|\psi(t)\rangle$ can now be written as:

$$|\Psi(t)\rangle = e^{\left(-\frac{i}{\hbar} Ht\right)} |\Psi(0)\rangle \quad (4.11)$$

The initial quantum state $|\psi(0)\rangle$ can be represented in the eigenbasis of H as:

$$|\Psi(0)\rangle = \sum_i a_i |E_i\rangle \quad (4.12)$$

where $H|E_i\rangle = E_i|E_i\rangle$ and a_i are time independent coefficients of $|e_i\rangle$. Now, substituting eq.(4.12) into the eq.(4.11), we obtain

$$|\Psi(t)\rangle = \sum_i a_i e^{-\frac{i}{\hbar} E_i t} |E_i\rangle \quad (4.13)$$

The imaginary time evolution[2] technique proceeds by evolving some initial state is imaginary time with the Hamiltonian of interest. This is done by changing $t \rightarrow -i\tau$ where τ is the imaginary time. The resultant quantum state tends to be in the ground state $|E_0\rangle$ for a long time

$$|E_0\rangle = \lim_{\tau \rightarrow \infty} \frac{e^{-\tau H} |\Psi(0)\rangle}{\sqrt{|\langle \Psi(\tau) | \Psi(\tau) \rangle|}} \quad (4.14)$$

where

$$|\Psi(\tau)\rangle = e^{-\tau H} |\Psi(0)\rangle \quad (4.15)$$

for some initial quantum state $|\Psi(0)\rangle$ that has non-zero overlap with the ground state. Hence, we can clearly see by comparing eq.(4.13) and eq.(4.15), the imaginary time evolution leads to ground states since E_0 is the smallest value and the other higher excited will decay faster compared to the ground state.

The time evolution working mechanism is described below:

- **Step 1:** We need to split the evolution operator into small imaginary-time steps as

$$e^{-\tau H} = (e^{\delta\tau H})^m \quad (4.16)$$

where $\delta\tau = \tau/m$ with $m \gg 1$.

Let us assume for the sake of simplicity that the Hamiltonian is the sum of two-body nearest-neighbour terms:

$$H = \sum_{\langle i,j \rangle} h_{ij} \quad (4.17)$$

where sites i and j are nearest neighbours in the lattice.

By applying the first-order Suzuki-Trotter expansion[10] (see Appendix A), we can make the approximation to first order in $\delta\tau$

$$e^{-\delta\tau H} \simeq \prod_{\langle i,j \rangle} e^{-\delta\tau h_{ij}} + \mathcal{O}(\delta\tau^2) \quad (4.18)$$

Further orders in this approximation can also be considered if necessary to reduce associated error. Each of the term in the multiplication above is called a two-body gate in the language of quantum circuits. The two-body gate g_{ij} between sites i and j is given by:

$$g_{ij} = e^{-\delta\tau h_{ij}} \quad (4.19)$$

Thus, the imaginary-time evolution operator can be approximated by repeated operations of the operator $m \gg 1$ times:

$$U(\delta\tau) \equiv \prod_{\langle i,j \rangle} g_{ij} \quad (4.20)$$

There are three ways to apply this operation on a given MPS, which is show below:

Once the above arrangements are made, we need to proceed with the following strategy:

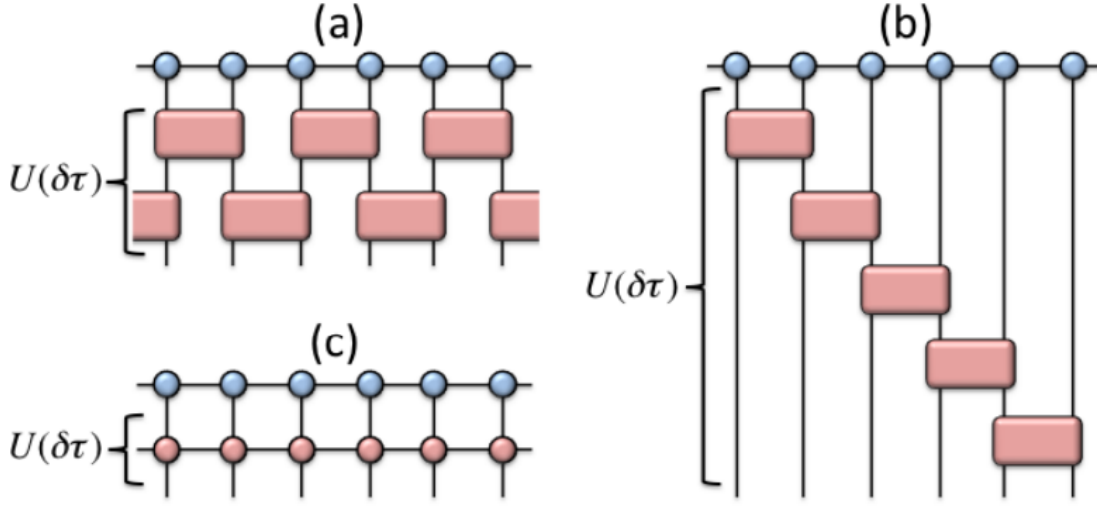


Figure 4.2: Different ways of applying $U(\delta\tau)$ on an MPS (a) Parallel, (b) Series and (c) Matrix Product Operator (MPO) (taken from Ref [2])

1. Infinitesimal evolution:

Apply operator $U(\delta\tau)$ on the MPS of the quantum state $|\Psi\rangle$ with dimension D , obtaining a new MPS state $|\tilde{\Psi}\rangle$ such that:

$$|\tilde{\Psi}\rangle = U(\delta\tau) |\Psi\rangle \quad (4.21)$$

with a new bond dimension $D' \geq D$.

2. Truncation:

Approximate $|\tilde{\Psi}\rangle$ by a new MPS $|\Psi'\rangle$ back with bond dimension D .

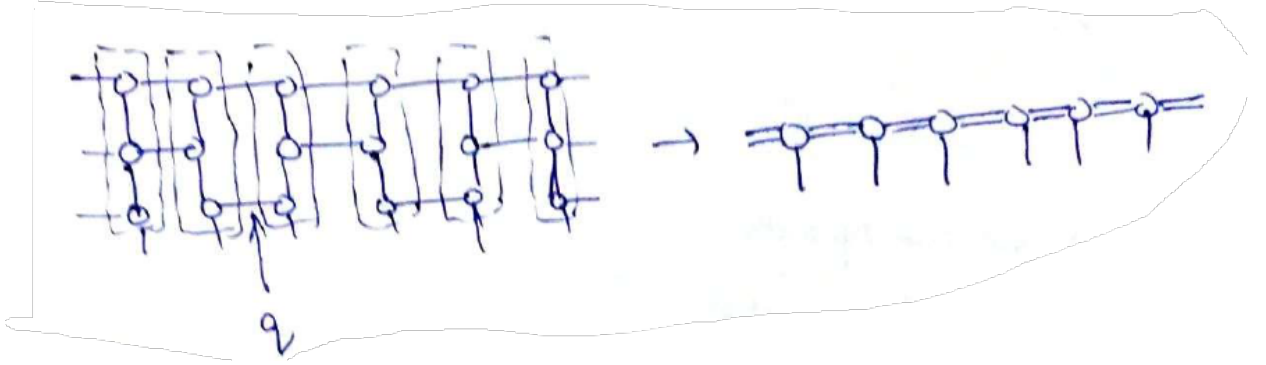


Figure 4.3: fig: Implementing step(1) by applying the evolution gates in parallel to some MPS with q being the bond dimension of g_{ij} . One obtains a new MPS with larger bond dimension.

- **Step 2:** We now perform the minimization of the squared distance error:

$$\text{err} = \left\| |\Psi'\rangle - |\tilde{\Psi}\rangle \right\|^2 \quad (4.22)$$

This minimization is done again using the tensor by tensor strategy i.e., we fix all the tensors in $|\Psi'\rangle$ except say tensor A . Then,

$$\begin{aligned} \min_A \left\| |\Psi'\rangle - |\tilde{\Psi}\rangle \right\|^2 &= \min_A \left| \langle \Psi' | \Psi' \rangle - \langle \tilde{\Psi} | \Psi' \rangle - \langle \Psi' | \tilde{\Psi} \rangle + \langle \tilde{\Psi} | \tilde{\Psi} \rangle \right| \\ &= \min_A \left(\vec{A}^\dagger \mathcal{N} \vec{A} - \vec{B}^\dagger \vec{A} - \vec{A}^\dagger \vec{B} + C \right) \end{aligned} \quad (4.23)$$

where $\vec{A}$ are the components of tensor A arranged as a vector, $\mathcal{N}$ is the normalization matrix, $\vec{B}$ are the components of vector corresponding to the environment of tensor A in $\langle \Psi' | \tilde{\Psi} \rangle$ and $C = \langle \tilde{\Psi} | \tilde{\Psi} \rangle$ is a scalar.

The minimization then proceeds as

$$\frac{\partial}{\partial \vec{A}^\dagger} \left(\vec{A}^\dagger \mathcal{N} \vec{A} - \vec{B}^\dagger \vec{A} - \vec{A}^\dagger \vec{B} + C \right) = 0 \quad (4.24)$$

Therefore, we obtain the condition

$$\vec{A} = \mathcal{N}^{-1} \vec{B} \quad (4.25)$$

Thus, once matrix $\mathcal{N}$ and the vector $\vec{B}$ are known, one needs to compute the above equation which is a system of linear equations.

In the step (2), the system of linear equations can be solved easily for small D . For large bond dimension, the complexity grows and one needs to use specialized numerical packages. The performing the entire process especially the step(2) completely, it normally called full update. Here the full environment of tensor A has been taken into account in order to update its components. Despite having a relatively large computational cost, this approach is also the most accurate one but more demanding for 2d systems.

4.2.1 Time Evolving Block Decimation

To reduce the computational cost of the full update, there is an alternate method known as simple update. It avoids the full calculation of the environment of tensor A during its update which is the bottleneck of the calculation. Instead, we use some simplified environment for which almost no calculation is needed. It leads to huge oversimplification of the effect of quantum correlations in the system. Most of the time it follows from some type of mean field approximation (or Bethe Lattice approximation). The TEBD algorithm alternates between applying local two-site evolution gates and truncating the resulting bond dimensions through singular value decomposition. By repeating these operations over successive imaginary-time steps, the MPS converges towards the ground state while maintaining a fixed bond dimension.

This method is really good for 1d systems as long as the infinitesimal time step $\delta\tau$ is sufficiently small. This method is known as Time-Evolving Block Decimation (TEBD) method.

However, the accuracy of the results drops down substantially when approaching a quantum critical point. Here the correlation length is large and the full effect of all the surrounding environment of a tensor is important.

4.3 Stability and the Normalization matrix $\mathcal{N}$

Although the variational optimization and imaginary-time evolution methods are based on different physical principles, both ultimately require solving linear systems or generalized eigenvalue problems involving the normalization matrix $\mathcal{N}$. Consequently, the numerical stability of these algorithms depends critically on the conditioning of $\mathcal{N}$. By definition, $\mathcal{N}$ is hermitian and positive since it can always be written as $\mathcal{N} = X^\dagger X$ for some matrix X . Therefore, all its eigenvalues are positive.

Taking this into account, there are two possible problems that could arise:

1. Even when computed exactly, the matrix may possess eigenvalues that are very close to zero.
2. When computed approximately, the matrix may acquire small negative eigenvalues because of numerical errors.

In either of the two cases, the corresponding solution to the generalized eigenvalue problem or linear system of the equations may produce wrong results. If not somehow corrected, the error usually amplifies over the iterations in the algorithm, hence killing the convergence and producing instabilities. Therefore, it is very important to compute matrices $\mathcal{N}$ as accurate as possible at every step and make it as well conditioned as possible.

Depending on the tensor network under consideration, several stabilization strategies may be employed.

1. For MPS with open boundary condition:

This TN has no loops, so it is always possible to choose an appropriate gauge for the tensors in which $\mathcal{N} = \mathbb{I}$. This is actually achieved by the canonical form for MPS with open boundary condition. The algorithm is thus more stable and the problem to solve is far more simple. However the cost of this method is that one needs to compute the canonical form at every step.

2. For MPS with periodic boundary condition

This TN has loops. One can think of adding/subtracting to $\mathcal{N}$ some infinitesimal constant term proportional to the identity with the aim to kill eigenvalues close to zero or slightly negative:

$$\mathcal{N} \rightarrow \mathcal{N} + \epsilon \mathbb{I} \text{ with } \epsilon \ll 1.$$

Other method is to make the matrix "as hermitian as possible" at every step by using

$$\mathcal{N} \rightarrow \frac{\mathcal{N} + \mathcal{N}^\dagger}{2}$$

instead of $\mathcal{N}$ as long as the introduced difference is small enough.

In a realistic situation one tries to combine all of the "stabilizing tricks" mentioned above in order to make the algorithm as stable as possible.

The numerical stability of tensor network algorithms is therefore closely connected to the gauge freedom of the MPS representation. Exploiting the canonical form whenever possible greatly improves both the efficiency and robustness of ground-state optimization algorithms. In the next chapter, these concepts are employed in practical numerical simulations using the ITensor library.

Chapter 5

Application of MPS to the Transverse Field Ising model for benchmarking

In the previous chapters, the theoretical framework of Matrix Product States and the Density Matrix Renormalization Group (DMRG) algorithm were discussed. In this work, the ground state of the one-dimensional Transverse Field Ising Model (TFIM) was obtained using the built-in DMRG algorithm for a finite size system. The objective was to benchmark the MPS representation by comparing physical observables across different values of the transverse magnetic field with high dimensional Hilbert space.

5.1 The Transverse Field Ising Model

The transverse Field Ising Model (TFIM)[11] is one of the simple models extensively used for the study of phase transitions in statistical mechanics as well as for quantum systems. We have considered our model to be a one dimensional spin chain with an open boundary condition described by the Hamiltonian:

$$H_{TFIM} = -J \sum_{i=1}^{N-1} \sigma_i^z \sigma_{i+1}^z - h \sum_{i=1}^N \sigma_i^x \quad (5.1)$$

where σ_i^z and σ_i^x are the Pauli Z and X operators respectively acting on the i^{th} particle, J is the interaction strength between the spin particles and h is the order parameter or strength of the external field provided which is along X-axis (the transverse field).

For the numerical simulation, we set the system size to be $N = 30$ which is typically hard to simulate using normal eigenvalue solver package. We set $J = 1$ and used h/J as the control parameter. We vary h/J such that it spans across both ferromagnetic ($h \ll J$) and paramagnetic ($h \gg J$) regimes including the critical region $h/J \approx 1$.

5.2 Benchmarking using ITensor

To demonstrate the practical effectiveness of these methods, numerical simulations were performed using the ITensors and ITensorMPS library in Julia. ITensor is a high-level tensor network library. It stands for “Intelligent Tensor”. It provides efficient tensor contractions, automatic index management, and optimized implementations of variational algorithms such as DMRG. These features allow complex quantum many-body systems to be studied with relatively compact and readable code.

The approach for benchmarking is done in the following sequence:

- i. We first define the number of sites required for the simulation using `siteinds[4][5]` function.
- ii. Using this, we construct the Hamiltonian using `opsum[4][5]` and `MPO[4][5]` functions.
- iii. We then initialize our matrix product state `random_mps[4][5]` function.
- iv. And using `DMRG[4][5]`, we compute the ground state by fixing number of sweeps.
- v. We used `orthogonalize[4][5]` and `SVD[4][5]` functions to evaluate entanglement entropy.
- vi. For magnetization, `expect[4][5]` function was used.

For greater details, all of these steps are discussed in the Appendix B.

5.3 Challenges faced in Benchmarking

For small system size DMRG performed well. But as the system size increases being an optimization algorithm it got stuck to some local minima. To solve this issue DMRG has parameter for the inclusion of noise which introduces random values while fixing the parameters of the MPS. We set the first half of the number sweeps to have noise which decreases with each sweep. This guaranteed the convergence of the DMRG to give accurate values of the ground state energy compared with respect to the results given by exact denationalization method. After making sure it converges well till the system size of 12 spin particles, we head for $N = 30$.

5.4 Magnetization and the GHZ states

To find the magnetization we used the formula:

$$M_z = \frac{1}{N} \sum_{i=1}^N \sigma_i^z \quad (5.2)$$

So, starting from $h = 0$, we varied it till $h = 2$, with the step size of $\Delta h = 0.025$ and used the ground of the previous iteration in the next iteration. This showed perfect phase transition as shown in fig. . But doing the reverse, we get an anomalous result. The state got into the ferromagnetic state from the paramagnetic state but the net magnetization was 0. So, in this case the magnetization did not serve us a good measure to see phase transitions. Whereas the entanglement entropy perfectly captured the phase transitions in both the cases but were different in these two cases.

The reason for this was as we move towards the ferromagnetic state from paramagnetic, DMRG captures degenerate ground states which has stronger superposition of both up and down states that is :

$$|\psi\rangle_{ground} \approx a |\uparrow\uparrow \dots \uparrow\rangle + b |\downarrow\downarrow \dots \downarrow\rangle \quad (5.3)$$

where a and b are complex/real coefficients such that $a^2 + b^2 = 1$. So, for GHZ states we can write:

$$|\psi\rangle_{ground} = \frac{1}{\sqrt{2}} (|\uparrow\uparrow \dots \uparrow\rangle + |\downarrow\downarrow \dots \downarrow\rangle) \quad (5.4)$$

and computing $\langle \psi | H_{TFIM} | \psi \rangle$ for the above GHZ state for ($h \ll J$) we have:

$$\begin{aligned} \langle \psi | H | \psi \rangle &= \langle \uparrow \uparrow \dots \uparrow | H_{TFIM} | \uparrow \uparrow \dots \uparrow \rangle \\ &= \langle \downarrow \downarrow \dots \downarrow | H_{TFIM} | \downarrow \downarrow \dots \downarrow \rangle \end{aligned} \quad (5.5)$$

Thus they are degenerate states of the ground state energy of H_{TFIM} .

In order to break this superposition, we introduce a longitudinal symmetry breaking field $\epsilon \sum_{i=1}^N \sigma_i^z$ in the H_{TFIM} such that:

$$H'_{TFIM} = H_{TFIM} - \epsilon \sum_{i=1}^N \sigma_i^z \quad (5.6)$$

where $\epsilon \ll 1$.

With this, we made DMRG to prefer the up state: $|\uparrow \uparrow \dots \uparrow\rangle$ and thus we get out from this superposition condition.

In classical simulation, where Monte Carlo method is used to randomly flip the spins based on the energy of the nearest neighbours, it does not require any kind of symmetry breaking field. In both the simulations we suffer from the finite size effect. Theoretically the transition should occur around $h = J$ but due to the finite size effect there is a shift.

5.5 Numerical Result

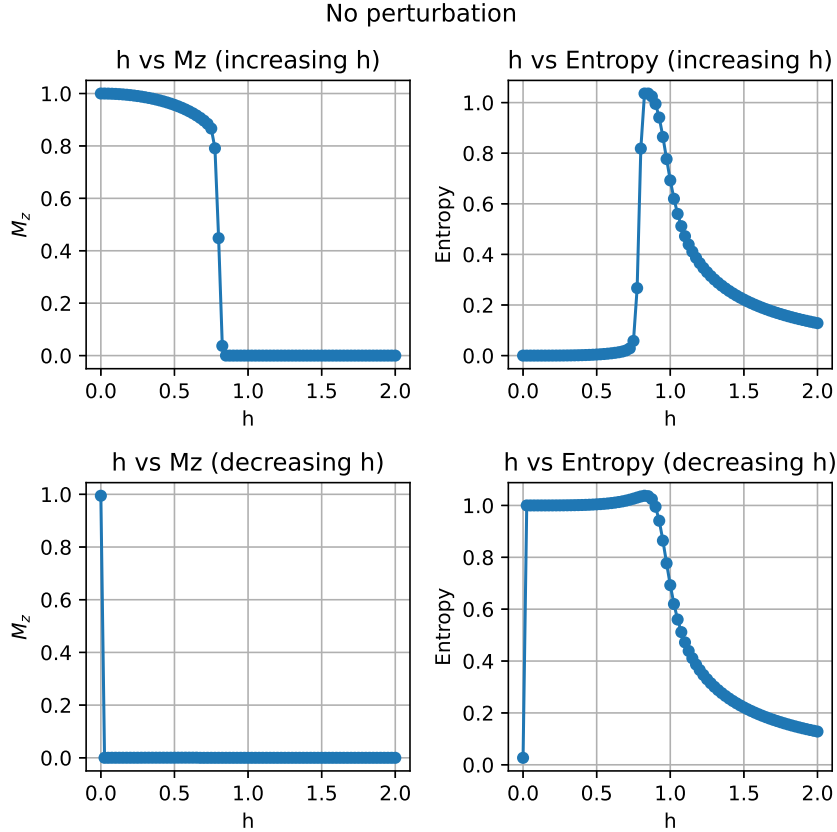


Figure 5.1: TFIM without symmetry breaking field

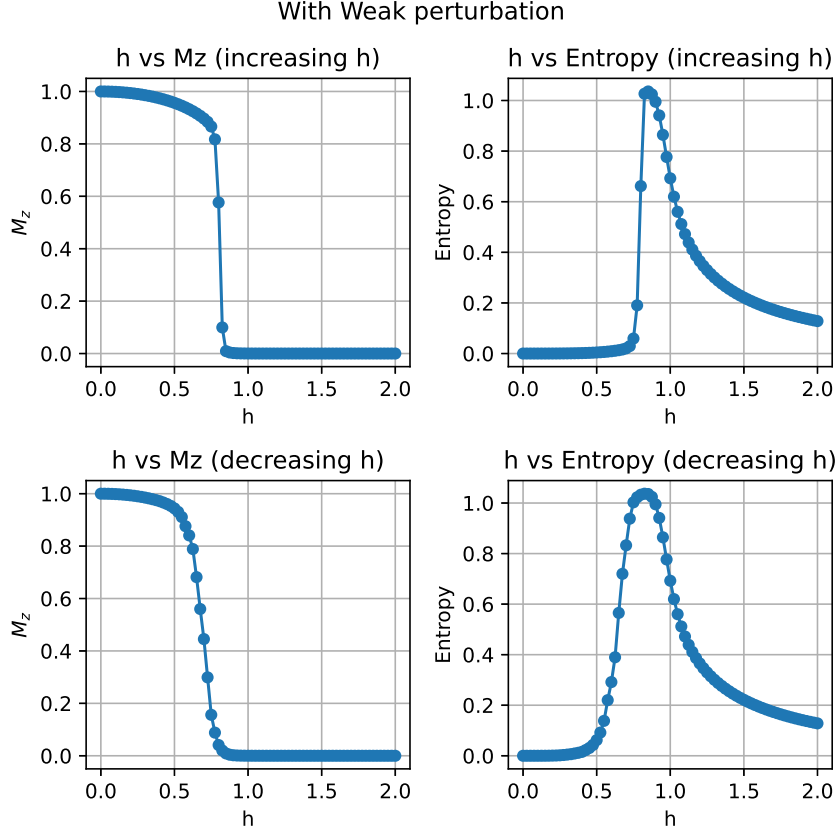


Figure 5.2: TFIM with a longitudinal symmetry breaking field with $\epsilon = 10^{-6}$

5.6 Discussion

The primary objective of the present numerical study was to benchmark the Matrix Product State representation using the DMRG algorithm implemented in ITensor. The one-dimensional Transverse Field Ising Model serves as an ideal testbed because its ground-state properties and quantum phase transition are well understood. The numerical results obtained in this work are consistent with the expected theoretical behaviour.

The ground-state energy decreases smoothly with the transverse magnetic field and exhibits the expected behaviour across the quantum phase transition. The absence of discontinuities confirms that the transition is continuous (second-order), consistent with the exact solution of the TFIM.

The longitudinal magnetization obtained without an explicit symmetry-breaking field remains close to zero in the ferromagnetic region. This behaviour is not a failure of the DMRG algorithm but rather a consequence of the finite-size ground state preserving the $\mathbb{Z}_2$ spin-flip symmetry. Consequently, the optimized state is a GHZ-like superposition of the two symmetry-broken ferromagnetic states.

Introducing a small longitudinal magnetic field removes the degeneracy of the ferromagnetic ground state and selects one of the symmetry-broken configurations. The resulting magnetization clearly distinguishes the ordered and disordered phases, allowing the phase transition to be identified more easily. This behaviour agrees with theoretical expectations and highlights the role of spontaneous symmetry breaking in finite systems.

The benchmarking also illustrates the efficiency of Matrix Product States for one-dimensional quantum systems obeying the area law of entanglement. The DMRG algorithm converged rapidly for most values of the transverse field, particularly when adiabatic continuation and

appropriate noise scheduling were employed. These observations demonstrate the robustness of the MPS representation for studying gapped one-dimensional quantum many-body systems.

Overall, the numerical results validate the theoretical concepts discussed in the previous chapters and demonstrate that Matrix Product States, together with the DMRG algorithm, provide an accurate and computationally efficient framework for obtaining the ground states of strongly correlated one-dimensional quantum systems.

Appendix A

Suzuki Trotter Expansion

The Suzuki–Trotter decomposition plays a central role in the Time-Evolving Block Decimation (TEBD) algorithm discussed in Chapter 4. While the application of the decomposition to tensor network algorithms follows the presentation of Orús [2], the detailed derivation presented in this appendix is adapted from Ref. [10] with notation modified to remain consistent throughout this report.

Derivation for first-order expansion

Let H be an operator such that $H = H_A + H_B$ where H_A and H_B are non-commuting operators. Now, the operator e^H has the form:

$$e^{-\tau H} = e^{-\tau(H_A + H_B)} \quad (\text{A.1})$$

And, by Taylor expansion of e^X for some operator X , we have:

$$e^X = \mathbb{I} + X + \frac{1}{2!}X^2 + \frac{1}{3!}X^3 + \dots \quad (\text{A.2})$$

Substituting eq.A.2 into eq.A.1, we get

$$e^{-\tau(H_A + H_B)} = \mathbb{I} - \tau(H_A + H_B) + \frac{\tau^2}{2!}(H_A + H_B)^2 + \dots \quad (\text{A.3})$$

Thus, we can observe that $(H_A + H_B)^2 = H_A^2 + H_A H_B + H_B H_A + H_B^2$ since $[H_A, H_B] \neq 0$. Now, taking the product $e^{H_A} e^{H_B}$, we have

$$\begin{aligned} e^{-\tau H_A} e^{-\tau H_B} &= (\mathbb{I} - \tau H_A + \frac{\tau^2}{2!}H_A^2 + \dots)(\mathbb{I} - \tau H_B + \frac{\tau^2}{2!}H_B^2 + \dots) \\ &= \mathbb{I} - \tau(H_A + H_B) + \frac{\tau^2}{2!}(H_A^2 + 2H_A H_B + H_B^2) + \dots \end{aligned} \quad (\text{A.4})$$

Thus we see that $e^{(H_A + H_B)} \neq e^{H_A} e^{H_B}$ for $[H_A, H_B] \neq 0$.

Consider dividing τ into n parts, such that

$$e^{-\tau H} = (e^{-\tau/n H})^n = (e^{-\delta\tau H})^n \quad (\text{A.5})$$

For $n \gg 1$, we have the first order approximation of $e^{H/n}$ as:

$$e^{-\delta\tau H} \approx \mathbb{I} - \delta\tau H + \mathcal{O}(\delta\tau^2) = \mathbb{I} - \delta\tau(H_A + H_B) + \mathcal{O}(\delta\tau^2) \quad (\text{A.6})$$

Similarly, dividing the product e^{H_A} and e^{H_B} into n parts each, we get the first order approximation

$$\begin{aligned} (e^{-\tau/n H_A} e^{-\tau/n H_B}) &\approx (\mathbb{I} - \tau/n H_A)(\mathbb{I} - \tau/n H_B) \\ &\approx \mathbb{I} - \delta\tau(H_A + H_B) + \mathcal{O}(1/n^2) \end{aligned} \quad (\text{A.7})$$

Thus, from eq.A.6 and eq.A.7, we can deduce that

$$e^{-\delta\tau H} \approx e^{-\delta\tau H_A} e^{-\delta\tau H_B} \quad (\text{A.8})$$

But we also have the Binomial expansion property that:

$$e^{(-X)} = \lim_{N \rightarrow \infty} (1 - X/N)^N \quad (\text{A.9})$$

Thus, we can conclude that:

$$\lim_{n \rightarrow \infty} (e^{-\tau/n H})^n = e^H = \lim_{n \rightarrow \infty} (e^{(-\tau/n H_A)} e^{(-\tau/n H_B)})^n \quad (\text{A.10})$$

Appendix B

Implementing ITensor in Julia

B.1 Construction of the Matrix Product Operator

The Hamiltonian of the transverse field Ising model was constructed using the `OpSum` interface and converted into a Matrix Product Operator (MPO) before applying DMRG.

```
1  using ITensors
2  using ITensorMPS
3
4  sites = siteinds("S=1/2", N)
5
6  Op = OpSum()
7
8  for i in 1:N-1
9      Op += -J, "Z", i, "Z", i+1
10     Op += -h, "X", i
11 end
12
13 Op += -h, "X", N
14
15 # Small symmetry breaking field
16 for i in 1:N
17     Op += -1e-6, "Z", i
18 end
19
20 H = MPO(ampo, sites)
```

B2. Construction of MPS and DMRG

The MPS is constructed by providing the same sites given to construct the Hamiltonian into `random_mps`

```
1
2  psi0 = random_mps(sites)
3  noise = [1e-3, 1e-4, 1e-5, 1e-6, 0]
4  energy, psi = dmrg(
5  H,
6  psi0;
7  nsweeps = 20,
8  maxdim = [10, 20, 100, 200, 200],
```

```

9     cutoff = 1e-14,
10     noise = noise
11 )

```

In the above code:

- The cutoff is used to set the convergence limit
- nsweeps is the number of times we need to run DMRG to reach the desired converged value
- maxdim is the limit of bond dimension for each sweep and it takes the last value in maxdim list in further sweeps.
- noise introduces random fluctuations with the magnitudes provided in the list.

B3. Expectation Value

Computing expectation is very in ITensor.

```

1     mz = expect(psi, "Z")
2     magnetization = sum(mz)/length(psi)

```

B4. Entropy

To evaluate entropy we need to perform orthogonalization of the MPS using the Mixed Canonical form and then perform SVD:

```

1     psi = orthogonalize(psi, l) # l is the MPS site
2
3     U,S,V = svd(psi[l], (linkinds(psi,l-1)..., siteinds(psi,l)...))
4     SvN = 0.0 # Von Neumann entropy S_{vN}
5     for n=1:dim(S,1)
6         p = S[n,n]^2
7         SvN -= p*log(p)/log(2.0)
8     end

```

Bibliography

- [1] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 6 2012.
- [2] Román Orús. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117–158, October 2014.
- [3] Ian P McCulloch. From density-matrix renormalization group to matrix product states. *Journal of Statistical Mechanics: Theory and Experiment*, 2007(10):P10014–P10014, October 2007.
- [4] Matthew Fishman, Steven R. White, and E. Miles Stoudenmire. The ITensor Software Library for Tensor Network Calculations. *SciPost Phys. Codebases*, page 4, 2022.
- [5] Matthew Fishman, Steven R. White, and E. Miles Stoudenmire. Codebase release 0.3 for ITensor. *SciPost Phys. Codebases*, pages 4–r0.3, 2022.
- [6] D.C. Kay. *Schaum’s Outline of Tensor Calculus*. Schaum’s Outline Series. McGraw-Hill Companies, Incorporated, 1988.
- [7] Niilo Heikkinen. The canonical forms of matrix product states in infinite-dimensional hilbert spaces. *Journal of Physics A: Mathematical and Theoretical*, 58(32):325201, August 2025.
- [8] Kelvin Koor, Yixian Qiu, Leong Chuan Kwek, and Patrick Rebentrost. A short tutorial on wirtinger calculus with applications in quantum information, 2023.
- [9] N. Zettili. *Quantum Mechanics: Concepts and Applications*. Wiley, 2009.
- [10] TILOCK SADHUKHAN. Suzuki-Trotter Decomposition.pdf. 2 2025.
- [11] Rudraksh Sharma. Quantum phase transitions in the transverse-field ising model: A comparative study of exact, variational, and hardware-based approaches, 2026.